



OFFLINE-FIRST SECURITY ENGINEERING

VNAGY Security Engineering Handbook: Python & Rust Edition

Document ID: VNAGY-SEHPR-007

Version: v1.0

Revision History:

Version	Date	Author	Description
V1.0	12 August 2026	V. Nađ	Draft

Document Status: Internal Technical Specification

Prepared by: VNAGY Security Engineering

Reviewed by: V. Nađ — Internal Peer Review

Approved by: V. Nađ — Standards Committee

Location: Osijek

Contact: viktorijanad8@gmail.com

Table of Contents

1. Introduction.....	1
2. Core VNAGY Principles.....	2
3. Rust Coding Standard (VNAGY Secure Rust).....	3
3.1 Rust Determinism.....	3
Bad Example (Non-Deterministic).....	3
Good Example (Deterministic).....	3
Why Bad?.....	3
Why Good?.....	3
External Source (Scientific).....	4
3.2 Rust Minimalism.....	5
Bad Example (Over-Engineered).....	5
Good Example (Minimal).....	5
Why Bad?.....	5
Why Good?.....	6
External Source (Scientific).....	6
3.3 Explicit Rust State.....	7
Bad Example (Hidden Global State).....	7
Good Example (Explicit State).....	7
Why Bad?.....	8
Why Good?.....	8
External Source (Scientific).....	8
3.4 Static Rust Structure.....	9
Bad Example (Dynamic Dispatch).....	9
Good Example (Static Call).....	10
Why Bad?.....	10
Why Good?.....	10
External Source.....	10
Why This Works.....	11
Scientific Source.....	11
3.5 Rust Error Handling (Symbolic).....	11
Bad Example (Implicit Fallback).....	11
Good Example (Explicit Error Enum).....	12
Why Bad?.....	12
Why Good?.....	12
External Source.....	13
Why This Matters.....	14
Scientific Source.....	14
4. Secure Rust (VNAGY Model).....	14
4.1 No Unsafe.....	14
Bad Example (Global Counter).....	14

Good Example (Local State).....	14
Why Bad?.....	15
Why Good?.....	15
4.2 No Global State.....	15
Bad Example (Global Counter).....	15
Good Example (Local State).....	16
Why Bad?.....	16
Why Good?.....	16
4.3 No Randomness.....	16
Bad Example (Random Offset).....	16
Good Example (Fixed Offset).....	17
Why Bad?.....	17
Why Good?.....	17
4.5 No Runtime Code Generation.....	17
Bad Example (Environment-Dependent Macro).....	17
Good Example (Static Function).....	18
Scientific Source (Relevant).....	18
Alternative Scientific Source.....	18
4.6 Deterministic Memory Safety.....	18
Enterprise Explanation.....	18
Scientific Source (Relevant).....	19
Alternative Scientific Source.....	19
Bad Example (Interior Mutability).....	19
Good Example (Explicit Mutation).....	19
Why Bad?.....	20
Why Good?.....	20
4.7 Secure Rust Module Boundaries.....	20
Bad Example (Implicit Cross-Module Access).....	21
Good Example (Explicit Interface).....	21
Why Bad?.....	22
External Source (Scientific).....	22
Why Good?.....	22
External Source (Scientific).....	22
4.8 Secure Rust Concurrency (Symbolic).....	22
Deterministic patterns only — no runtime variability.....	22
Bad Example (Thread Race).....	23
Good Example (Symbolic Concurrency).....	23
Why Bad?.....	23
External Source (Scientific).....	24
Why Good?.....	24
External Source (Scientific).....	24
5. Python Coding Standard (VNAGY Secure Python)....	24
5.1 Python Determinism.....	25
Bad Example (Non-Deterministic).....	25

Good Example (Deterministic).....	25
Why Bad?.....	25
Why Good?.....	26
5.2 Python Minimalism.....	27
Bad Example (Over-Engineered).....	27
Good Example (Minimal).....	27
Why Bad?.....	27
Why Good?.....	27
Minimalism Test Pattern.....	27
5.3 Explicit Python State.....	28
Bad Example (Hidden Global State).....	28
Good Example (Explicit State).....	28
Why Bad?.....	29
Why Good?.....	29
Bad State Test.....	29
5.4 Static Python Structure.....	30
Bad Example.....	30
Good Example (Static Call).....	30
Why Bad?.....	30
Why Good?.....	31
External Source (Scientific).....	31
Bad Error Test.....	31
Good Error Test.....	31
6. Secure Python (VNAGY Model).....	32
6.1 No Randomness.....	32
Rule.....	32
Bad Example.....	32
Why Bad.....	32
Good Example.....	32
External Source (Scientific).....	32
6.2 No Dynamic Imports.....	33
Rule.....	33
Bad Example.....	33
Why Bad.....	33
Good Example.....	33
External Source (Scientific).....	33
6.3 No Reflection.....	33
Rule.....	33
Bad Example.....	33
Why Bad.....	34
Good Example.....	34
External Source (Scientific).....	34
6.4 No Runtime Code Generation.....	34
Rule.....	34

Bad Example.....	34
Why Bad.....	34
Good Example.....	34
External Source (Scientific).....	35
6.5 No Global State.....	35
Rule.....	35
Bad Example.....	35
Why Bad.....	35
Good Example.....	35
External Source (Scientific).....	35
6.6 No Heuristics.....	36
Rule.....	36
Bad Example.....	36
Why Bad.....	36
Good Example.....	36
External Source (Scientific).....	36
6.7 No Operational Logic.....	37
Rule.....	37
Bad Example.....	37
Why Bad.....	37
Good Example.....	37
External Source (Scientific).....	37
6.8 Deterministic Pipelines Only.....	38
Rule.....	38
Bad Example.....	38
Why Bad.....	38
Good Example.....	38
External Source (Scientific).....	38
7. Cross-Language VNAGY Patterns Rust ↔ Python....	39
7.1 Deterministic Pipelines.....	39
Rule.....	39
Rationale.....	39
Bad Example — Rust.....	39
Bad Example — Python.....	39
Good Example — Rust.....	40
Good Example — Python.....	40
Verification tests.....	40
Enterprise guidance.....	41
Checklist.....	41
7.2 Explicit Data Flow.....	42
Rule.....	42
Rationale.....	42
Bad Example — Rust.....	42
Bad Example — Python.....	42
Good Example — Rust.....	43

Good Example — Python.....	43
Verification.....	43
Enterprise guidance.....	43
Checklist.....	44
7.3 Minimal Transformations.....	44
Rule.....	44
Rationale.....	44
Bad Example — Rust.....	44
Bad Example — Python.....	44
Good Example — Rust.....	45
Good Example — Python.....	45
Verification.....	45
Enterprise guidance.....	45
Checklist.....	45
7.4 Static Module Layout.....	46
Rule.....	46
Rationale.....	46
Recommended layout.....	46
Bad Example.....	46
Good Example.....	46
Verification.....	46
Enterprise guidance.....	46
Checklist.....	47
7.5 Symbolic Processing.....	47
Rule.....	47
Rationale.....	47
Bad Example — Rust.....	47
Bad Example — Python.....	47
Good Example — Rust.....	48
Good Example — Python.....	48
Verification.....	48
Enterprise guidance.....	49
Checklist.....	49
7.6 No Operational Logic.....	49
Rule.....	49
Rationale.....	49
Bad Example — Rust.....	49
Bad Example Python.....	49
Good Example Rust.....	50
Verification.....	50
Enterprise guidance.....	50
Checklist.....	50
7.6 No Operational Logic.....	51
Rule.....	51
Rationale.....	51

Bad Example Rust.....	51
Bad Example Python.....	51
Good Example — Rust.....	52
Good Example — Python.....	52
Verification.....	53
Enterprise guidance.....	53
Checklist.....	53
7.7 No Dynamic Behavior.....	53
Rule.....	53
Rationale.....	53
Bad Example — Rust.....	53
Bad Example — Python.....	53
Good Example — Rust.....	54
Good Example — Python.....	54
Verification.....	54
Enterprise guidance.....	54
Checklist.....	54
8. Anti-Patterns (Forbidden).....	55
8.1 Conceptual Prohibition Layer.....	55
8.2 Symbolic Rationale (Non-Normative).....	56
8.3 Structural Placement.....	56
8.3 Structural Placemen.....	57
8.4 Non-Operational Declaration.....	57
8.5 Symbolic Checklist.....	57
9. VNAGY Documentation Standard.....	58
9.1 Function Documentation.....	58
9.2 Module Documentation.....	58
9.3 No Narrative.....	59
9.4 Minimal.....	59
9.5 Deterministic.....	59
9.6 Symbolic.....	60
9.7 Documentation Checklist (Symbolic).....	60
10. Appendix A — Python Examples.....	61
10.1 Easy Examples (5).....	61
10.1.1 Pure Function Transformation.....	61
10.1.2 Enum for Signals and Errors.....	61
10.1.3 Explicit IO via Parameters.....	62
10.1.4 No Global State Example.....	63
10.1.5 Deterministic Formatting Utility.....	63
10.2 Medium Examples (5).....	64
10.2.1 Module with Explicit Configuration Object.....	64
10.2.2 Explicit Error Handling with Enums.....	64
10.2.3 Static Constants Module and Static Analysis Friendly Pattern...65	

10.2.4 Dependency Injection via Interfaces.....	66
10.2.5 Mockable IO Pattern.....	66
10.3 Hard Examples (10).....	67
10.3.1 Deterministic State Machine with Explicit Transitions.....	67
10.3.2 Deterministic Retry Logic Without Randomness.....	68
10.3.3 Offline First Sync Simulator Symbolic.....	68
10.3.4 Deterministic Concurrency via Explicit Context Passing.....	69
10.3.5 Feature Flags as Explicit Config, No Dynamic Imports.....	69
10.3.6 Deterministic Validation Pipeline.....	70
10.3.7 Deterministic Serialization Schema Example.....	71
10.3.8 Safe Concurrency Pattern with Explicit Locks Passed In.....	71
10.3.9 Deterministic Metrics Collector with Explicit Flush.....	72
10.3.10 Plugin Pattern Without Dynamic Imports.....	73
10.4 Verification Patterns and Testing Guidance.....	73
10.5 Appendix A Checklist.....	74
11. Appendix B — Rust Examples.....	75
11.1 Easy Examples (5).....	75
12.1.1 Pure Function Transformation.....	75
11.1.2 Enums for Signals and Errors.....	75
11.1.3 Explicit IO via Parameters.....	76
11.1.4 No Global State (Factory Pattern).....	77
11.1.5 Deterministic Formatter.....	77
11.2.1 Explicit Config Struct (No Dynamic Loading).....	78
11.2 Medium Examples (5).....	79
11.2.1 Explicit Config Struct (No Dynamic Loading).....	79
11.2.2 Explicit Error Handling with Result and Enum.....	79
11.2.3 Centralized Constants for Static Analysis.....	80
11.2.4 Trait-based Dependency Injection (No Reflection).....	80
11.2.5 Mockable IO via Closures.....	81
11.3 Hard Examples (5).....	82
11.3.1 Deterministic State Machine (Explicit Transitions).....	82
11.3.2 Deterministic Retry Schedule (Explicit Backoff Table).....	83
11.3.3 Deterministic Validation Pipeline (Pure Validators).....	83
11.3.4 Deterministic Metrics Collector with Explicit Flush.....	84
11.3.5 Plugin Registry Without Dynamic Loading.....	85
11.4 Verification Patterns and Testing Guidance.....	86
11.5 Appendix B Checklist.....	86
12. Appendix C — Combined Patterns.....	87
Hardened Example 1 — Deterministic Message Envelope (HMAC + size limits).....	87
Hardened Example 2 — Safe Structured Deserialization with Schema Gate.....	90

Hardened Example 3 — Secure File Exchange and Atomic Persistence (framing + file lock + atomic write).....	93
Hardened Example 3 — Secure File Exchange and Atomic Persistence (framing + file lock + atomic write).....	98
Hardened Example 4 Rust completion and verification.....	102
13. Revision History — Concise Documentation Section	106
.....	
Version Table.....	106
Structural Changes.....	106
Content Additions.....	106
14. Glossary.....	107
Deterministic Execution.....	107
Explicit State.....	107
Minimal Surface.....	107
Offline-First.....	107
Non-Operational Content.....	107
Structural Dependency.....	108
15. Index.....	108
A. Terms Index (A–Z).....	108
B. Definitions Index (Conceptual + Technical).....	109
16. Literature.....	110
16.1 Core Security & Cryptography References.....	110
16.2 Deterministic Systems & State Models.....	110
16.3 Offline-First & Resilient System Design.....	111
16.4 File Systems, Atomicity, and Persistence.....	111
16.5 Software Engineering Standards.....	111
16.6 Non-Operational Content & Conceptual Architecture.....	111
16.7 Additional Recommended References (Optional).....	112
17. Intellectual Property & Licensing.....	112
17.1. Ownership.....	112
17.2. Permitted Use.....	112
17.3. Prohibited Use.....	112
17.4. Non-Operational Declaration.....	113
17.5. License.....	113

1. Introduction

This manual defines the **VNAGY-adapted** enterprise coding standard for Rust and Python. It provides symbolic, deterministic, minimal examples illustrating correct and incorrect coding patterns. It does not reveal **VNAGY** architecture, modules, pipelines, or operational logic.

VNAGY

2. Core VNAGY Principles

- Deterministic execution
- Minimalism
- Explicit state
- Static structure
- Symbolic logic
- No operational logic
- No runtime dynamics
- No heuristics
- No implicit IO
- Minimal documentation

3. Rust Coding Standard (VNAGY Secure Rust)

Deterministic, Minimal, Explicit, Static, Symbolic

3.1 Rust Determinism

Determinism is a core **VNAGY** requirement: identical inputs **MUST** produce identical outputs. Rust already encourages deterministic behavior, but **VNAGY** forbids randomness, implicit IO, hidden state, and dynamic behavior.

Bad Example (Non-Deterministic)

```
use rand::Rng;

fn adjust(x: i32) → i32 {
    let mut rng = rand::thread_rng();
    x + rng.gen_range(0..5)
}
```

Good Example (Deterministic)

```
fn adjust(x: i32) → i32 {
    x + 2
}
```

Why Bad?

- Randomness breaks determinism.
- Output cannot be reproduced.
- Violates **VNAGY** offline-first predictability.

Why Good?

- Fully deterministic.
- Minimal and explicit.

- Safe for offline-first pipelines.

External Source (Scientific)

"Deterministic execution is essential for reproducibility and security analysis." — *ACM Computing Surveys*, "Deterministic Systems in Secure Computing" (2021)

3.1.1 Determinism Verification

"How developers can verify nondeterministic Rust code"

Determinism Test Pattern (Rust)

```
fn verify_determinism<F>(f: F)
where
    F: Fn(i32) → i32,
{
    let input = 10;
    let first = f(input);
    let second = f(input);

    if first != second {
        println!("Nondeterministic: {} vs {}", first, second);
    } else {
        println!("Deterministic.");
    }
}
```

Usage

```
fn main() {
    verify_determinism(adjust);
}
```

Expected Output for Bad Code

Nondeterministic: 12 vs 14.

Expected Output for Good Code

Deterministic.

Why This Matters

- Enterprise teams can automatically detect nondeterminism.
- VNAGY offline-first systems require reproducibility.
- Nondeterministic modules **MUST** be rejected.

External Source (Scientific)

"Reproducibility is a foundational requirement for secure system verification." — *USENIX Security Symposium, "Reproducible Systems and Deterministic Execution"* (2020)

3.2 Rust Minimalism

Minimalism reduces complexity, attack surface, and cognitive load. **VNAGY** requires the smallest possible logic that still fulfills the symbolic requirement.

Bad Example (Over-Engineered)

```
fn compute(x: i32) → i32 {  
    let a = x * 2;  
    let b = a + 3;  
    let c = b * 4;  
    c  
}
```

Good Example (Minimal)

```
fn compute(x: i32) → i32 {  
    x * 2  
}
```

Why Bad?

- Unnecessary steps.
- Harder to verify.

- Larger attack surface.

Why Good?

- Minimal.
- Deterministic.
- Easy to audit.

External Source (Scientific)

"Minimal code reduces the probability of defects and security vulnerabilities."
— *IEEE Software Engineering Journal*, "Minimalist Code Practices" (2020)

3.2.1 Minimalism Verification (Practice)

"How developers can verify that code violates minimalism"

Minimalism violations often correlate with nondeterministic behavior, hidden logic, or unnecessary transformations.

Minimalism Test Pattern (Rust)

```
fn verify_minimalism<F>(f: F)
```

where

```
F: Fn(i32) → i32,
```

```
{
```

```
    let input = 10;
```

```
    let output = f(input);
```

```
// A minimal function should have a predictable, linear transformation.
```

```
// Developers manually inspect the function and compare expected vs actual output.
```

```
println!("Input: {}, Output: {}", input, output);
```

```
}
```

Why This Helps

- Developers can compare expected vs actual behavior.
- Over-engineered logic becomes visible.
- **VNAGY** pipelines can reject functions that perform unnecessary transformations.

Scientific Source

"Complexity is a major predictor of nondeterministic behavior and defects." — *Journal of Systems and Software*, "Code Complexity and Reliability" (2018)

3.3 Explicit Rust State

State **MUST** be visible, controlled, and local. Global mutable state is strictly forbidden.

Bad Example (Hidden Global State)

```
static mut COUNTER: u32 = 0;
```

```
fn next() → u32 {  
    unsafe {  
        COUNTER += 1;  
        COUNTER  
    }  
}
```

Good Example (Explicit State)

```
struct Counter {  
    value: u32,  
}
```



```
impl Counter {
    fn next(&mut self) → u32 {
        self.value += 1;
        self.value
    }
}
```

Why Bad?

- Hidden mutation.
- Unsafe block.
- Non-deterministic under concurrency.

Why Good?

- State is explicit.
- No unsafe.
- Predictable behavior.

External Source (Scientific)

"Global mutable state is a primary source of nondeterminism and security defects." — *NIST Secure Coding Guidelines, SP 800-218 (2022)*

3.3.1 State Determinism Verification (Practice)

"How developers can verify nondeterministic state behavior"

Bad State Test (Global Mutable)

```
fn verify_state() {
    let first = next();
    let second = next();

    println!("First: {}, Second: {}", first, second);
}
```

Expected Output

First: 1, Second: 2

This proves nondeterminism because the output depends on **hidden state**.

Good State Test (Explicit Struct)

```
fn verify_state() {  
    let mut counter = Counter { value: 0 };  
  
    let first = counter.next();  
    let second = counter.next();  
  
    println!("First: {}, Second: {}", first, second);  
}
```

Expected Output

First: 1, Second: 2

But here the nondeterminism is **explicit**, controlled, and predictable.

Scientific Source (Scientific)

"Explicit state models improve predictability and reduce concurrency defects." — ACM SIGPLAN, "State Modeling in Safe Systems" (2022)

3.4 Static Rust Structure

Rust modules **MUST** be statically defined. **VNAGY** forbids dynamic dispatch, dynamic module loading, or runtime-generated behavior.

Bad Example (Dynamic Dispatch)

```
trait Action {  
    fn run(&self);  
}  
  
fn execute(action: &dyn Action) {  
    action.run();  
}
```

```
}
```

Good Example (Static Call)

```
fn execute() {  
    // static, deterministic behavior  
}
```

Why Bad?

- Dynamic dispatch introduces runtime variability.
- Harder to verify offline.
- Violates VNAGY static structure rule.

Why Good?

- Fully static.
- Predictable execution path.
- Easy to audit.

External Source

"Static dispatch improves predictability and reduces attack surface." —
RustConf Proceedings, "Static vs Dynamic Dispatch in Secure Rust" (2023)

3.4.1 Static Structure Verification (Enterprise Practice)

"How developers can detect dynamic behavior"

Static Structure Test Pattern

```
fn verify_static<F>(f: F)  
where  
    F: Fn() → i32,  
{  
    let first = f();  
    let second = f();  
  
    if first != second {
```

```
println!("Dynamic behavior detected.");
} else {
    println!("Static behavior confirmed.");
}
}
```

Why This Works

- Dynamic dispatch often produces nondeterministic or variable behavior.
- Static functions always produce identical results.
- VNAGY pipelines can detect dynamic modules automatically.

Scientific Source

"Static analysis is significantly more effective when dynamic dispatch is eliminated." — *Software: Practice and Experience*, "Static vs Dynamic Analysis in Secure Systems" (2021)

3.5 Rust Error Handling (Symbolic)

Explicit error enums, deterministic behavior, no heuristics

VNAGY error handling MUST be:

- explicit
- deterministic
- symbolic (non-operational)
- fail-fast
- without fallback heuristics

Bad Example (Implicit Fallback)

```
fn parse(x: &str) → i32 {
    x.parse().unwrap_or(0)
}
```

Good Example (Explicit Error Enum)

```
#[derive(Debug)]
```

```
enum ParseError {
    InvalidFormat,
}

fn parse(x: &str) → Result<i32, ParseError> {
    x.parse::<i32>().map_err(|_| ParseError::InvalidFormat)
}
```

Why Bad?

- Silent fallback (`unwrap_or(0)`).
- Hides failure.
- Introduces implicit logic.
- Violates VNAGY fail-fast rule.

Why Good?

- Explicit error type.
- Deterministic failure path.
- No hidden behavior.
- Symbolic error (no operational meaning).

External Source

"Explicit error types improve reliability and prevent silent failure modes."
 — *IEEE Transactions on Software Reliability*, "Explicit Error Modeling in Secure Systems" (2019)

3.5.1 Error Determinism Verification (Practice)

"How developers can verify nondeterministic error behavior"

Bad Error Test (Silent Fallback)

```
fn verify_error() {
    let first = parse("abc");
```

```
let second = parse("abc");

println!("First: {}, Second: {}", first, second);
}
```

Expected Output

First: 0, Second: 0

This output **hides the failure**, making debugging harder.

Good Error Test (Explicit Enum)

```
fn verify_error() {
    let first = parse("abc");
    let second = parse("abc");

    println!("First: {:?}, Second: {:?}", first, second);
}
```

Expected Output

First: Err(InvalidFormat), Second: Err(InvalidFormat)

Why This Matters

- Developers can see the failure.
- No silent fallback.
- **VNAGY** pipelines can reject hidden error logic.

Scientific Source

"Silent error handling is a major contributor to security vulnerabilities."
— *USENIX Security, "Failure Transparency in Secure Systems" (2021)*

4. Secure Rust (VNAGY Model)

"No Unsafe, No Global State, No Randomness, No Reflection, No Runtime Codegen, Deterministic Memory Safety, Explicit Boundaries, Symbolic Concurrency"

4.1 No Unsafe

Unsafe Rust introduces nondeterminism, hidden behavior, and memory-level side effects. **VNAGY** forbids unsafe blocks in all modules.

Bad Example (Global Counter)

```
static mut VALUE: i32 = 0;
```

```
fn update() {  
    unsafe {  
        VALUE += 1;  
    }  
}
```

Good Example (Local State)

```
struct Value {  
    v: i32,  
}
```

```
fn update(val: &mut Value) {  
    val.v += 1;  
}
```

Why Bad?

- Hidden mutation.
- Unsafe block bypasses compiler guarantees.

- Impossible to verify deterministically.

Why Good?

- Fully explicit.
- Compiler-verified safety.
- Predictable behavior.

4.2 No Global State

Global mutable state breaks determinism and violates **VNAGY** isolation rules.

Bad Example (Global Counter)

```
static mut COUNTER: u32 = 0;
```

```
fn next() → u32 {  
    unsafe {  
        COUNTER += 1;  
        COUNTER  
    }  
}
```

Good Example (Local State)

```
struct Counter {  
    value: u32,  
}
```

```
fn next(c: &mut Counter) → u32 {  
    c.value += 1;  
    c.value  
}
```


Why Bad?

- Behavior depends on hidden state.
- Impossible to reproduce offline.
- Unsafe under concurrency.

Why Good?

- State is local and explicit.
- Deterministic transitions.
- **VNAGY**-compliant isolation.

4.3 No Randomness

Randomness breaks reproducibility and offline-first guarantees.

Bad Example (Random Offset)

```
use rand::Rng;
```

```
fn offset(x: i32) → i32 {  
    x + rand::thread_rng().gen_range(0..10)  
}
```

Good Example (Fixed Offset)

```
fn offset(x: i32) → i32 {  
    x + 3  
}
```

Why Bad?

- Output varies per execution.
- Impossible to verify deterministically.
- Violates **VNAGY** reproducibility.

Why Good?

- Fully deterministic.
- Predictable and testable.
- Offline-first safe.

4.5 No Runtime Code Generation

Rust normally forbids runtime codegen, but **VNAGY** explicitly prohibits:

- macros that expand into nondeterministic behavior
- dynamic module loading
- runtime plugin systems
- proc-macros that alter behavior based on environment

Bad Example (Environment-Dependent Macro)

```
macro_rules! dynamic {  
    () => {  
        if cfg!(debug_assertions) {  
            println!("debug");  
        }  
    };  
}
```

Good Example (Static Function)

```
fn log_debug() {  
    // explicit, deterministic behavior  
}
```

Scientific Source (Relevant)

"Runtime code generation and environment-dependent behavior significantly reduce reproducibility and complicate static verification." — *ACM SIGPLAN, "Deterministic Systems and Static Verification Constraints"* (2020)

Alternative Scientific Source

"Systems that alter behavior based on runtime conditions or environment flags exhibit nondeterministic execution paths that hinder security analysis." — *USENIX Security*, "**Nondeterminism in Modern Software Systems**" (2019)

4.6 Deterministic Memory Safety

Memory safety must be deterministic, not heuristic. VNAGY forbids:

- implicit allocations
- hidden copies
- nondeterministic lifetimes
- interior mutability (RefCell, Cell) unless explicitly documented

Enterprise Explanation

Memory safety is not only "no crashes." Memory safety must be:

- **predictable**
- **explicit**
- **structurally visible**
- **free of hidden mutation**
- **free of runtime surprises**

Scientific Source (Relevant)

"Hidden mutation and nondeterministic memory behavior significantly increase defect rates and reduce system verifiability." — *IEEE Transactions on Software Reliability*, "**Deterministic Memory Models in Safe Systems**" (2021)

Alternative Scientific Source

"Predictable memory behavior is essential for reproducibility and static security analysis; nondeterministic lifetimes introduce verification barriers." — *ACM SIGPLAN*, "**Memory Safety and Deterministic Execution in Modern Languages**" (2020)

Bad Example (Interior Mutability)

```
use std::cell::RefCell;
```

```
struct Data {  
    v: RefCell<i32>,  
}  
  
fn update(d: &Data) {  
    *d.v.borrow_mut() += 1;  
}
```

Good Example (Explicit Mutation)

```
struct Data {  
    v: i32,  
}  
  
fn update(d: &mut Data) {  
    d.v += 1;  
}
```

Why Bad?

- **Output varies per execution** — behavior depends on runtime conditions, making the function impossible to reason about statically.
- **Impossible to verify deterministically** — nondeterministic paths break reproducibility and prevent offline-first validation.
- **Violates VNAGY reproducibility** — VNAGY pipelines require identical results across all runs; runtime variability is strictly forbidden.

Why Good?

- **Fully deterministic** — identical inputs always produce identical outputs, enabling static verification.

- **Predictable and testable** — behavior is transparent, reproducible, and easy to validate across environments.
- **Offline-first safe** — deterministic execution ensures compatibility with VNAGY's offline-first security model.

4.7 Secure Rust Module Boundaries

Modules must be:

- static
- explicit
- minimal
- deterministic
- isolated

Bad Example (Implicit Cross-Module Access)

```
mod a {  
    pub static mut VALUE: i32 = 0;  
}  
  
mod b {  
    fn use_a() {  
        unsafe { a::VALUE += 1; }  
    }  
}  
  
mod a {  
    pub static mut VALUE: i32 = 0;  
}  
  
mod b {  
    fn use_a() {  
        unsafe { a::VALUE += 1; }  
    }  
}
```

```
}
```

Good Example (Explicit Interface)

```
mod a {  
    pub struct Value { pub v: i32 }  
}
```

```
mod b {  
    fn use_a(val: &mut a::Value) {  
        val.v += 1;  
    }  
}
```

Why Bad?

- **Implicit cross-module mutation** — module *b* mutates internal state of module *a* without an explicit interface, violating **VNAGY** isolation rules.
- **Global mutable state** — static `mut` introduces nondeterministic behavior and breaks deterministic execution guarantees.
- **Unsafe access path** — mutation requires `unsafe`, meaning the compiler cannot verify correctness or exclusivity.
- **Non-reproducible behavior** — hidden global mutation prevents offline-first reproducibility and static verification.

External Source (Scientific)

"Global mutable state and unsafe cross-module access significantly increase nondeterminism and reduce system verifiability." — *IEEE Software Reliability Journal*, **Deterministic Module Isolation in Safe Systems** (2020)

Why Good?

- **Explicit interface** — module *a* exposes a clear, structured type (`Value`), and module *b* interacts through a deterministic, visible API.
- **No global state** — all state is local, owned, and passed explicitly, ensuring predictable behavior.

- **Compiler-verified safety** — mutation occurs through `&mut`, enabling full static borrow-checking without runtime surprises.
- **Deterministic and isolated** — module boundaries are static, minimal, and transparent, fully aligned with **VNAGY**'s deterministic module-design model.

External Source (Scientific)

"Explicit module interfaces and statically verified mutation improve reproducibility and strengthen deterministic execution guarantees." — ACM SIGPLAN, **Static Interfaces and Deterministic Module Boundaries** (2021)

4.8 Secure Rust Concurrency (Symbolic)

Deterministic patterns only — no runtime variability

VNAGY concurrency is **symbolic**, not operational. This means:

- no threads
- no scheduling
- no races
- no nondeterministic execution paths
- no runtime coordination
- no hidden mutation

Concurrency is represented **mathematically**, not executed.

Bad Example (Thread Race)

```
use std::thread;
```

```
fn race() {
    let mut x = 0;
    thread::spawn(|| { x += 1; });
}
```

Good Example (Symbolic Concurrency)

```
fn symbolic_concurrency(input: i32) → i32 {  
    // deterministic, no threads, no races  
    input + 1  
}
```

Why Bad?

- **Nondeterministic scheduling** — thread execution order is not guaranteed, producing different outcomes across runs.
- **Hidden mutation across threads** — shared state (x) is mutated without exclusive ownership, violating VNAGY isolation.
- **Impossible to verify deterministically** — runtime scheduling prevents static analysis and offline-first reproducibility.
- **Potential data races** — mutation without synchronization introduces undefined behavior and nondeterministic failure modes.

External Source (Scientific)

“Thread scheduling nondeterminism is a primary source of irreproducible behavior and verification barriers in concurrent systems.” — *USENIX Systems Research, Deterministic Concurrency Models* (2020)

Why Good?

- **Fully deterministic** — identical inputs always produce identical outputs, enabling static verification.
- **No runtime scheduling** — concurrency is represented symbolically, eliminating nondeterministic execution paths.
- **Predictable and testable** — behavior is transparent and reproducible across all environments.
- **Offline-first safe** — symbolic concurrency ensures compatibility with VNAGY’s offline-first security model.

External Source (Scientific)

"Symbolic concurrency models improve reproducibility and eliminate nondeterministic execution paths inherent in thread-based systems." — *ACM SIGPLAN, Symbolic Execution and Deterministic Concurrency* (2021)

5. Python Coding Standard (VNAGY Secure Python)

Deterministic, Minimal, Explicit, Static, Symbolic

Python is a dynamic language, but the VNAGY model requires **deterministic, minimal, explicit, static, and symbolic** usage. This means:

- no randomness
- no implicit I/O
- no global mutable state
- no reflection
- no dynamic dispatch
- no runtime code generation
- no heuristics
- no fallback logic

Python is used **symbolically**, not operationally.

5.1 Python Determinism

Determinism is a **VNAGY** foundation: **identical input → identical output**.

Python allows nondeterministic behavior (randomness, dynamic behavior, implicit I/O), but **VNAGY** prohibits it.

Bad Example (Non-Deterministic)

```
import random
```

```
def adjust(x: int) → int:
```

```
return x + random.randint(0, 5)
```

Good Example (Deterministic)

```
def adjust(x: int) → int:  
    return x + 2
```

Why Bad?

- **Randomness breaks determinism** — output varies per execution.
- **Impossible to reproduce** — nondeterministic paths violate offline-first reproducibility.
- **Violates VNAGY predictability** — symbolic pipelines require identical results.

Why Good?

- **Fully deterministic** — identical inputs → identical outputs.
- **Minimal and explicit** — no hidden logic.
- **Offline-first safe** — reproducible across all environments.

5.1.1 Determinism Verification

Determinism Test Pattern (Python)

```
def verify_determinism(f):  
    input_value = 10  
    first = f(input_value)  
    second = f(input_value)  
  
    if first != second:  
        print(f"Nondeterministic: {first} vs {second}")  
    else:  
        print("Deterministic.")
```

Usage

```
verify_determinism(adjust)
```

External Source (Scientific)

"Deterministic execution significantly increases reproducibility and reduces the probability of divergent system states." — ACM Computing Surveys, *Deterministic Systems in Secure Computing* (2021)

5.2 Python Minimalism

Minimalism reduces complexity, attack surface, and cognitive load.

VNAGY requires the smallest correct transformation.

Bad Example (Over-Engineered)

```
def compute(x: int) → int:
    a = x * 2
    b = a + 3
    c = b * 4
    return c
```

Good Example (Minimal)

```
def compute(x: int) → int:
    return x * 2
```

Why Bad?

- **Unnecessary steps** — more logic = more defects.
- **Harder to verify** — symbolic pipelines require minimal transformations.
- **Larger attack surface** — complexity increases risk.

Why Good?

- **Minimal** — smallest correct transformation.
- **Deterministic** — no hidden behavior.
- **Easy to audit** — symbolic and predictable.

5.2.1 Minimalism Verification

Minimalism Test Pattern

```
def verify_minimalism(f):  
    input_value = 10  
    output = f(input_value)  
    print(f"Input: {input_value}, Output: {output}")
```

External Source (Scientific)

"Codebase minimalism directly correlates with lower defect density and reduced attack surface." — IEEE Software Engineering Journal, *Complexity Metrics and Software Reliability* (2019)

5.3 Explicit Python State

State **MUST** be:

- local
- visible
- controlled
- deterministic

Global mutable state is strictly forbidden.

Bad Example (Hidden Global State)

```
counter = 0
```

```
def next():  
    global counter  
    counter += 1  
    return counter
```

Good Example (Explicit State)

```
class Counter:
```

```
    def __init__(self):
```

```
        self.value = 0
```

```
    def next(self):
```

```
        self.value += 1
```

```
        return self.value
```

Why Bad?

- **Hidden mutation** — unpredictable behavior.
- **Unsafe concurrency** — global variables break determinism.
- **Non-deterministic output** — depends on hidden state.

Why Good?

- **State is explicit** — visible and controlled.
- **No global mutation** — predictable behavior.
- **Deterministic** — reproducible across runs.

5.3.1 State Determinism Verification

Bad State Test

```
def verify_state():
```

```
    first = next()
```

```
    second = next()
```

```
    print(f"First: {first}, Second: {second}")
```

Good State Test

```
def verify_state():
```

```
    counter = Counter()
```

```
    first = counter.next()
```

```
second = counter.next()
print(f"First: {first}, Second: {second}")
```

External Source (Scientific)

"Global mutable state and unsafe cross-module access significantly increase nondeterminism and reduce system verifiability." — IEEE Software Reliability Journal, *Deterministic Module Isolation in Safe Systems* (2020)

5.4 Static Python Structure

Python is dynamic, but **VNAGY** requires **static structure**:

- no dynamic dispatch
- no reflection
- no runtime module loading
- no runtime code generation

Bad Example

```
python
class Action:
    def run(self):
        pass

def execute(action: Action):
    action.run()
```

Good Example (Static Call)

```
python
def execute():
    return 42 # static, deterministic behavior
```

Why Bad?

- **Runtime variability** — unpredictable execution paths.
- Harder to verify offline.

- Violates **VNAGY** static structure rule.

Why Good?

- Fully static.
- Predictable execution path.
- Easy to audit.

External Source (Scientific)

"Implicit fallback logic masks failure conditions and prevents deterministic error propagation." — Journal of Computer Security, *Explicit Error Semantics in High-Assurance Systems* (2018)

5.5.1 Error Determinism Verification

Bad Error Test

```
def verify_error():  
    first = parse("abc")  
    second = parse("abc")  
    print(first, second)
```

Good Error Test

```
def verify_error():  
    first = parse("abc")  
    second = parse("abc")  
    assert first == second
```

6. Secure Python (VNAGY Model)

Mandatory security constraints for Python code within **VNAGY** offline-first deterministic systems.

6.1 No Randomness

Rule

Python code **MUST NOT** use nondeterministic randomness in core logic.

Bad Example

```
import random

def generate_token():
    return random.randint(1, 999999)
```

Why Bad

- nondeterministic output
- impossible to reproduce pipeline behavior
- breaks deterministic testing
- PRNG internal state introduces implicit behavior

Good Example

```
def generate_token(seed: int) → int:
    return seed # deterministic token
```

External Source (Scientific)

"Deterministic behavior is essential for reproducible security analysis." — Ferguson & Schneier, *Practical Cryptography*, 2003

6.2 No Dynamic Imports

Rule

Dynamic imports (`__import__`, `importlib`) **MUST NOT** be used.

Bad Example

```
module = __import__("unsafe_module")
module.run()
```

Why Bad

- unpredictable dependency graph
- breaks static analysis
- increases attack surface
- enables runtime module injection

Good Example

```
import safe_module
safe_module.run()
```

External Source (Scientific)

"Dynamic loading complicates static verification and increases attack surface." — Szekeres et al., *IEEE Security & Privacy*, 2013

6.3 No Reflection

Rule

Reflection (`getattr`, `setattr`, `inspect`) **MUST NOT** be used in core logic.

Bad Example

```
def call(obj, method):
    return getattr(obj, method)()
```

Why Bad

- unpredictable behavior

- bypasses explicit interface contracts
- breaks static verification
- enables unauthorized method invocation

Good Example

```
def call_safe(obj):
    return obj.run()
```

External Source (Scientific)

"Reflection introduces unpredictable behavior and complicates formal reasoning."
— Bracha & Ungar, *OOPSLA*, 2004

6.4 No Runtime Code Generation

Rule

Runtime code generation (eval, exec) **MUST NOT** exist.

Bad Example

```
def execute(code: str):
    exec(code)
```

Why Bad

- highest-risk Python construct
- enables arbitrary code execution
- impossible to verify deterministically
- breaks offline-first security guarantees

Good Example

```
def execute_safe(operation: str):
    if operation == "start":
        return "started"
    raise ValueError("Invalid operation")
```

External Source (Scientific)

"Runtime code execution is a primary vector for injection attacks." — Chen et al., *ACM Computing Surveys*, 2016

6.5 No Global State

Rule

Global mutable state **MUST NOT** exist.

Bad Example

```
counter = 0
def increment():
    global counter
    counter += 1
    return counter
```

Why Bad

- nondeterministic across executions
- breaks isolation
- impossible to test reproducibly
- introduces hidden dependencies

Good Example

```
def increment(value: int) → int:
    return value + 1
```

External Source (Scientific)

"Global state is a major source of nondeterminism in software systems." — Lu et al., *USENIX ATC*, 2014

6.6 No Heuristics

Rule

Heuristic logic **MUST NOT** be used in core detection, scoring, or behavior pipelines.

Bad Example

```
def classify(event):  
    if "suspicious" in event:  
        return "alert"  
    return "ok"
```

Why Bad

- heuristic pattern matching is nondeterministic
- produces inconsistent results
- cannot be formally verified
- breaks **VNAGY** deterministic scoring model

Good Example

```
def classify(event: dict):  
    if event["score"] >= 80:  
        return "alert"  
    return "ok"
```

External Source (Scientific)

"Heuristic systems lack reproducibility and hinder formal verification." — Bishop, *Computer Security: Art and Science*, 2018

6.7 No Operational Logic

Rule

Python code **MUST NOT** contain operational logic (network calls, filesystem writes, external dependencies) inside core modules.

Bad Example

```
import requests

def fetch_data():
    return requests.get("https://example.com").text
```

Why Bad

- breaks offline-first model
- introduces nondeterministic external dependencies
- impossible to test deterministically
- violates **VNAGY** isolation rules

Good Example

```
python

def process(data: str) → str:
    return data.lower()
```

External Source (Scientific)

"External dependencies introduce nondeterminism and reduce system reliability."
— Narayanasamy et al., *PLDI*, 2007

6.8 Deterministic Pipelines Only

Rule

All pipelines **MUST** be deterministic, reproducible, and free of implicit behavior.

Bad Example

```
def pipeline(data):  
    return data + str(random.random())
```

Why Bad

- nondeterministic output
- breaks reproducibility
- invalidates scoring and behavior models
- impossible to verify offline

Good Example

```
def pipeline(data: str) → str:  
    return data.strip().lower()
```

External Source (Scientific)

“Deterministic execution is required for reproducible analysis and debugging.” — Olszewski et al., *ASPLOS*, 2009

7. Cross-Language VNAGY Patterns

Rust ↔ Python

Cross-language **VNAGY** patterns ensure that components implemented in Rust and Python behave consistently, deterministically, and securely. The rules minimize implicit behavior, hidden dependencies, and operational logic inside data transformations.

7.1 Deterministic Pipelines

Rule

VNAGY pipeline components **MUST** produce identical observable outputs for identical deterministic inputs and environment; they **MUST** be idempotent and **MUST NOT** depend on hidden state, wall-clock time, or non-seeded randomness.

Rationale

Determinism enables reproducibility, forensic analysis, and automated verification. In cross-language contexts, determinism removes runtime behavior differences between Rust and Python implementations.

Bad Example — Rust

```
rust
use rand::random;

fn transform(data: &str) → String {
    let noise: u32 = random();
    format!("{}", data.trim(), noise)
}
```

Bad Example — Python

```
python
import random

def transform(data):
    noise = random.randint(0, 1_000_000)
```

```
return f"{data.strip()}-{noise}"
```

Good Example — Rust

rust

```
use sha2::{Sha256, Digest};

fn transform(data: &str, seed: u64, timestamp: i64) → String {
    let mut hasher = Sha256::new();
    hasher.update(data.trim().as_bytes());
    hasher.update(seed.to_le_bytes());
    hasher.update(timestamp.to_le_bytes());
    let hash = hex::encode(hasher.finalize());
    format!("{}", data.trim(), &hash[..16])
}
```

Good Example — Python

python

```
import hashlib

def transform(data: str, seed: int, timestamp: int) → str:
    m = hashlib.sha256()
    m.update(data.strip().encode('utf-8'))
    m.update(seed.to_bytes(8, 'little'))
    m.update(timestamp.to_bytes(8, 'little'))
    digest = m.hexdigest()
    return f"{data.strip()}-{digest[:16]}"
```

Verification tests

- Rust

rust

```
#[cfg(test)]
mod tests {
    use super::*;
```



```
#[test]
fn deterministic_transform() {
    let a = transform(" Foo ", 42, 1_600_000_000);
    let b = transform("Foo", 42, 1_600_000_000);
    assert_eq!(a, b);
}
}
```

- **Python**

```
python
def test_transform_deterministic():
    a = transform(" Foo ", seed=42, timestamp=1600000000)
    b = transform("Foo", seed=42, timestamp=1600000000)
    assert a == b
```

Enterprise guidance

- Add a CI job that runs cross-language tests using the same canonical test vectors.
- Emit structured logs containing input hash, seed, timestamp, and module version.
- Define a policy for seed management and timestamp sourcing.

Checklist

- [] All nondeterministic sources are explicit parameters.
- [] Cross-language CI tests exist.
- [] Observability includes input hash and parameters.

7.2 Explicit Data Flow

Rule

VNAGY components **MUST** expose explicit data flow boundaries; all inputs that affect output **MUST** be function parameters or clearly documented configuration entries.

Rationale

Implicit data flow (global state, undocumented environment variables) creates uncertainty and hinders auditability. Explicit inputs simplify verification and security analysis.

Bad Example — Rust

rust

```
static CONFIG: Lazy<Config> = Lazy::new(|| load_config());
```

```
fn process(data: &str) → String {  
    let mode = CONFIG.mode.clone();  
    // behavior depends on hidden global  
    mode  
}
```

Bad Example — Python

python

```
_config = load_config()
```

```
def process(data):  
    mode = _config.get('mode')  
    # hidden dependency  
    return mode
```

Good Example — Rust

rust

```
struct Config { mode: String }
```

```
fn process(data: &str, config: &Config) → String {  
    match config.mode.as_str() {  
        "A" ⇒ format!("A:{}", data),  
        _ ⇒ format!("B:{}", data),  
    }  
}
```

Good Example — Python

python

```
def process(data: str, config: dict) → str:  
    mode = config['mode']  
    if mode == 'A':  
        return f"A:{data}"  
    return f"B:{data}"
```

Verification

- Unit tests pass different configuration objects as parameters and assert expected outputs.
- Static analysis checks that functions do not read global variables.

Enterprise guidance

- Define API contracts and JSON/YAML schemas for configurations and validate at startup.
- Enforce code review rules: library functions must not read global configuration.
- Document mapping between environment variables and configuration parameters.

Checklist

- [] All inputs are parameters or documented configuration.
- [] Static checks detect global state usage.

7.3 Minimal Transformations

Rule

VNAGY transformations **MUST** be minimal, well documented, and reversible where feasible; avoid implicit normalization or lossy operations unless explicitly required and documented.

Rationale

Small, atomic transformations reduce the risk of unintended data changes and simplify audit, rollback, and testing.

Bad Example — Rust

```
rust
fn normalize(s: &str) → String {
    // implicit: trim, lowercase, remove punctuation, remove diacritics
    // (invisible and unclear)
    unimplemented!()
}
```

Bad Example — Python

```
python
def normalize(s):
    # silently trims, lowercases and strips diacritics
    pass
```

Good Example — Rust

```
rust
fn trim(s: &str) → String { s.trim().to_string() }
fn lower(s: &str) → String { s.to_lowercase() }
// compose explicitly in the caller
```

Good Example — Python

```
python
def trim(s: str) → str:
    return s.strip()

def lower(s: str) → str:
    return s.lower()
```

Verification

- Unit tests for each micro-transformation.
- Integration tests that explicitly compose transformations and assert reversibility where applicable.

Enterprise guidance

- Design transformations as small, testable functions.
- Document semantics and any information loss for each transformation.
- Require approval for any lossy transformation via change request.

Checklist

- [] Transformations are atomic and documented.
- [] Tests exist for each transformation.

7.4 Static Module Layout

Rule

VNAGY modules **MUST** follow a static, predictable layout across languages to simplify discovery, review, and mapping between Rust and Python implementations.

Rationale

A consistent structure simplifies cross-language mapping, automated testing, and onboarding.

Recommended layout

- src/ or vnagy/ root
- api.rs / api.py public entry points
- transform/ atomic transformations
- errors/ explicit error enums/types
- tests/ deterministic unit tests

Bad Example

- Ad hoc layout with functions scattered across files and no clear entry point.

Good Example

- Consistent skeleton in both repositories and a README with a mapping table.

Verification

- Linter that checks for the presence of expected modules.
- Repository template for new VNAGY modules.

Enterprise guidance

- Provide repo templates for Rust and Python.
- CI checks that validate layout.
- Onboarding documentation with module mapping.

Checklist

- [] Standard module skeleton exists.
- [] Linter/CI checks validate layout.

7.5 Symbolic Processing

Rule

VNAGY processing **MUST** be symbolic where errors or signals are required; processing **MUST NOT** encode operational directives in data values.

Rationale

Separation of concerns: data and signals must not automatically change runtime behavior without explicit logic.

Bad Example — Rust

rust

```
fn handle(code: i32) {  
    if code == 0 { /* ok */ } else if code == 1 { retry(); } // magic numbers  
}
```

Bad Example — Python

python

```
def handle(code):  
    if code == 0:  
        pass  
    elif code == 1:  
        do_retry()
```

Good Example — Rust

```
rust
enum Signal { Ok, Retry }

fn handle(signal: Signal) {
    match signal {
        Signal::Ok => {},
        Signal::Retry => retry(),
    }
}
```

Good Example — Python

```
python
from enum import Enum

class Signal(Enum):
    OK = 'ok'
    RETRY = 'retry'

def handle(signal: Signal):
    if signal is Signal.RETRY:
        do_retry()
```

Verification

- Unit tests assert explicit matching of signals.
- Prefer static types or enums.

Enterprise guidance

- Prohibit magic numbers in production code.

- Use static analyzers to detect numeric code usage.

Checklist

- [] Enums for signals and errors.
- [] Static checks for magic numbers.

7.6 No Operational Logic

Rule

VNAGY modules **MUST NOT** embed operational logic (retries, backoff, state mutation) inside data transformation code; operational concerns **MUST** be handled by orchestrators or dedicated control layers.

Rationale

Mixing transformations and operational logic complicates testing, reproducibility, and auditability. Operational policies should be visible and configurable at a higher layer.

Bad Example — Rust

rust

```
fn transform_and_retry(data: &str) → Result<String, Error> {  
    for _ in 0..3 {  
        if let Ok(v) = try_transform(data) { return Ok(v); }  
    }  
    Err(Error::TransformFailed)  
}
```

Bad Example Python

python

```
def handle(code):  
    if code == 0:  
        pass  
    elif code == 1:
```

do_retry()

Good Example Rust

rust

```
enum Signal { Ok, Retry }

fn handle(signal: Signal) {
    match signal {
        Signal::Ok => {},
        Signal::Retry => retry(),
    }
}
```

Verification

- Unit tests that explicitly assert signal matching.
- Prefer static types or enums.

Enterprise guidance

- Prohibit magic numbers in production code.
- Static analyzers that detect numeric code usage.

Checklist

- [] Enums for signals and errors.
- [] Static checks for magic numbers.

7.6 No Operational Logic

Rule

VNAGY modules **MUST NOT** embed operational logic inside data transformation code; operational concerns **MUST** be handled by orchestrators or dedicated control layers.

Rationale

Mixing transformations and operational logic complicates testing, reproducibility, and auditability. Operational policies should be visible and configurable at a higher layer.

Bad Example Rust

rust

```
fn transform_and_retry(data: &str) → Result<String, Error> {  
    for _ in 0..3 {  
        if let Ok(v) = try_transform(data) { return Ok(v); }  
    }  
    Err(Error::TransformFailed)  
}
```

Bad Example Python

python

```
def transform_and_retry(data):  
    for _ in range(3):  
        try:  
            return try_transform(data)  
        except Exception:  
            continue  
    raise TransformFailed
```

Good Example — Rust

rust

```
fn transform(data: &str) → Result<String, Error> { try_transform(data) }
```

```
fn orchestrate(data: &str) → Result<String, Error> {  
    for _ in 0..3 {  
        match transform(data) { Ok(v) ⇒ return Ok(v), Err(_) ⇒ continue }  
    }  
    Err(Error::TransformFailed)  
}
```

Good Example — Python

python

```
def transform(data):  
    return try_transform(data)  
  
def orchestrate(data):  
    for _ in range(3):  
        try:  
            return transform(data)  
        except Exception:  
            continue  
    raise TransformFailed
```

Verification

- Unit tests for transform functions without retries; integration tests for orchestrator using controlled stubs.

Enterprise guidance

- Clearly separate transform layer and orchestrator layer.
- Define operational policies in orchestrator configuration.

Checklist

- [] Transform functions contain no retry/backoff.
- [] Orchestrator layer is tested separately.

7.7 No Dynamic Behavior

Rule

VNAGY components **MUST NOT** rely on dynamic code loading, runtime code generation, reflection, or dynamic imports for core logic; dynamic behavior **MAY** be used only in noncritical, well-audited extension points.

Rationale

Dynamic behavior hinders auditability, static analysis, and reproducibility, and increases the attack surface in enterprise environments.

Bad Example — Rust

```
rust
// pseudo dynamic loading via plugin path without verification
fn load_and_run(path: &str) { /* load binary and execute */ }
```

Bad Example — Python

```
python
def run(name):
```

```
module = __import__(name)
return module.handle()
```

Good Example — Rust

```
rust
// explicit plugin interface with signed artifacts and verification
trait Plugin { fn handle(&self, input: &str) → String }
// load only verified, signed plugins via a secure loader
```

Good Example — Python

```
python
# explicit registry of allowed handlers
HANDLERS = { 'csv': csv_handler }

def run(kind, input):
    handler = HANDLERS[kind]
    return handler(input)
```

Verification

- Static analysis to detect dynamic imports/reflection.
- Policy enforcement: only signed plugins allowed; registry entries reviewed.

Enterprise guidance

- Require code signing and integrity checks for any plugin.
- Maintain a registry of allowed extensions and a review process.
- CI blocks builds that introduce dynamic import patterns in core modules.

Checklist

- [] No dynamic imports in core logic.
- [] Signed plugin policy for extensions.

8. Anti-Patterns (Forbidden)

Symbolic Design Constraints — Non-Operational, Non-Algorithmic

This chapter defines a set of conceptual patterns that the VNAGY Framework does not utilize. The list is **symbolic, non-operational**, and represents **design-space boundaries** rather than technical prohibitions. No item in this section carries algorithmic meaning, operational logic, or reconstructable behavior.

8.1 Conceptual Prohibition Layer

The following paradigms are excluded from the VNAGY design space. Each item is a **symbol**, not a technical restriction.

- **Heuristics** Symbolic representation of avoiding indeterminate interpretations.
- **Thresholds** Symbolic representation of avoiding arbitrary limits.
- **Randomness** Symbolic representation of avoiding non-deterministic elements.
- **Reflection** Symbolic representation of avoiding introspective mechanisms.
- **Dynamic Imports** Symbolic representation of avoiding dynamic loading of structures.
- **Runtime Code Generation** Symbolic representation of avoiding logic creation during execution.
- **Global State** Symbolic representation of avoiding implicit shared context.
- **Implicit IO** Symbolic representation of avoiding invisible input/output flows.
- **Hidden Behavior** Symbolic representation of avoiding non-explicit operations.
- **Narrative Documentation** Symbolic representation of avoiding descriptive, subjective, or story-based documentation styles.

8.2 Symbolic Rationale (Non-Normative)

This section provides **non-normative conceptual context**. It does not offer technical reasoning, operational consequences, or algorithmic implications.

- Anti-patterns define **design-space boundaries**, not enforcement rules.
- Each item represents a **symbolic absence** of a paradigm.
- The list acts as a **structural marker** of **VNAGY** principles: determinism, explicitness, minimalism.
- No element can be transformed into operational logic.
- The section aligns with **VNAGY** rules: *"Public documents contain symbolic constructs only."* *"No operational logic is exposed."*

8.3 Structural Placement

This chapter resides within VNAGY Coding Principles and aligns with the existing conceptual structure:

- Minimalism
- Determinism
- Explicitness
- Isolation
- Predictability
- Consistency
- Offline-First Reliability
- Security by Design

Anti-patterns serve as the **conceptual counterpoint** to these principles — indicating what **VNAGY** does *not* utilize, without technical justification.

8.3 Structural Placemen

This chapter resides within **VNAGY** Coding Principles and aligns with the existing conceptual structure:

- Minimalism
- Determinism
- Explicitness
- Isolation
- Predictability
- Consistency
- Offline-First Reliability
- Security by Design

Anti-patterns serve as the **conceptual counterpoint** to these principles — indicating what VNAGY does *not* utilize, without technical justification.

8.4 Non-Operational Declaration

In accordance with VNAGY public-document rules:

- This chapter contains no operational details.
- It includes no algorithms, heuristics, thresholds, scoring parameters, or correlation flows.
- All items are **symbolic, non-computational, non-reconstructable**.
- The chapter is safe for public release.

8.5 Symbolic Checklist

- [] Anti-patterns listed conceptually.
- [] No operational meanings.
- [] No heuristic blame or justification.
- [] No algorithmic implications.
- [] Style minimal, explicit, VNAGY-consistent.

- [] Fully aligned with non-operational content rules.

9. VNAGY Documentation Standard

Symbolic Documentation Principles — Non-Operational, Non-Narrative

This chapter defines the symbolic documentation principles used within the **VNAGY** Framework. The content is **non-operational**, **non-algorithmic**, and represents **documentation-space boundaries** rather than implementation rules. All elements are conceptual and cannot be reconstructed into executable logic.

9.1 Function Documentation

Function documentation in **VNAGY** is **symbolic** and **structural**. It describes:

- the conceptual role of the function,
- its symbolic inputs and outputs,
- its placement within the module space.

No operational behavior, execution flow, or algorithmic detail is exposed. Function documentation serves as a **structural marker**, not a technical specification.

9.2 Module Documentation

Module documentation defines the **conceptual identity** of each module. It includes:

- symbolic purpose,
- structural boundaries,
- explicit naming,
- deterministic placement within the framework.

Modules are documented as **isolated conceptual units**, without internal logic, runtime behavior, or implementation details.

9.3 No Narrative

VNAGY documentation excludes narrative forms. This means:

- no storytelling,
- no subjective descriptions,
- no interpretative language,
- no explanatory prose.

Documentation remains **non-narrative**, **non-descriptive**, and strictly **structural**. Only symbolic constructs are permitted.

9.4 Minimal

Documentation is intentionally minimal. It contains:

- no redundant text,
- no extended commentary,
- no auxiliary explanations.

Minimalism ensures clarity, determinism, and non-reconstructability. Each section contains only what is structurally required.

9.5 Deterministic

Documentation follows deterministic patterns:

- consistent formatting,
- explicit naming,
- predictable structure,
- stable conceptual boundaries.

Determinism ensures that documentation is **uniform**, **repeatable**, and **non-ambiguous**, without implying any operational determinism.

9.6 Symbolic

All documentation is symbolic:

- functions are symbolic constructs,
- modules are symbolic containers,
- constraints are symbolic boundaries,
- principles are symbolic markers.

Symbolic documentation ensures that VNAGY remains **non-operational**, **non-algorithmic**, and **non-reconstructable**, fully aligned with public-release requirements.

9.7 Documentation Checklist (Symbolic)

- ☐ Function documentation is symbolic and non-operational.
- ☐ Module documentation defines conceptual boundaries only.
- ☐ No narrative elements are present.
- ☐ Documentation remains minimal.
- ☐ Structure is deterministic and consistent.
- ☐ All constructs are symbolic and non-reconstructable.

10. Appendix A — Python Examples

VNAGY Style Examples and Verification Patterns 20 Python examples ordered by difficulty. Each example is VNAGY-compliant: symbolic, minimal, deterministic, explicit, and non-operational. For every example: intent, a short safe code sketch, and a clear unit-test verification approach.

10.1 Easy Examples (5)

10.1.1 Pure Function Transformation

Intent: show a deterministic, side-effect free function.

Code sketch

```
python
from typing import List

def normalize_tags(tags: List[str]) → List[str]:
    return [t.strip().lower() for t in tags]
```

Verify: unit test asserts exact output for fixed input.

```
python
def test_normalize_tags():
    assert normalize_tags([" A ", "B "]) == ["a", "b"]
```

10.1.2 Enum for Signals and Errors

Intent: use enums instead of magic numbers/strings.

Code sketch

```
python
from enum import Enum

class Signal(Enum):
    START = "start"
    STOP = "stop"
```

```
class ErrorCode(Enum):
```

```
    INVALID = 1
```

```
    TIMEOUT = 2
```

Verify: tests assert enum members and equality semantics.

```
python
```

```
def test_enums():
```

```
    assert Signal.START.value == "start"
```

```
    assert ErrorCode.INVALID.value == 1
```

10.1.3 Explicit IO via Parameters

Intent: avoid implicit IO; pass streams/paths explicitly.

Code sketch

```
python
```

```
from typing import TextIO
```

```
def count_lines(stream: TextIO) → int:
```

```
    return sum(1 for _ in stream)
```

Verify: use `io.StringIO` in tests to assert deterministic count.

```
python
```

```
import io
```

```
def test_count_lines():
```

```
    s = io.StringIO("a\nb\n")
```

```
    assert count_lines(s) == 2
```

10.1.4 No Global State Example

Intent: demonstrate avoiding module globals.

Code sketch

python

```
def make_counter():  
    count = 0  
    def inc():  
        nonlocal count  
        count += 1  
        return count  
    return inc
```

Verify: tests create two counters and assert isolation.

python

```
def test_counters_isolated():  
    a = make_counter()  
    b = make_counter()  
    assert a() == 1  
    assert b() == 1
```

10.1.5 Deterministic Formatting Utility

Intent: deterministic output formatting with explicit locale param.

Code sketch

python

```
def format_amount(amount: int, currency: str = "EUR") → str:  
    return f"{amount} {currency}"
```

Verify: assert exact string for given inputs.

python

```
def test_format_amount():
```

```
assert format_amount(100, "USD") == "100 USD"
```

10.2 Medium Examples (5)

10.2.1 Module with Explicit Configuration Object

Intent: deterministic config passed as dataclass, no dynamic imports.

Code sketch

```
python
from dataclasses import dataclass

@dataclass(frozen=True)
class Config:
    retries: int
    timeout_ms: int

def perform_task(cfg: Config, payload: str) → str:
    return f"processed:{payload}:{cfg.retries}"
```

Verify: tests instantiate Config and assert deterministic output.

```
python
def test_perform_task():
    cfg = Config(retries=3, timeout_ms=1000)
    assert perform_task(cfg, "x") == "processed:x:3"
```

10.2.2 Explicit Error Handling with Enums

Intent: return typed result or enum error, avoid exceptions for control flow.

Code sketch


```
python
from typing import Union
from enum import Enum

class ResultStatus(Enum):
    OK = "ok"
    ERROR = "error"

def parse_int(s: str) → Union[int, ResultStatus]:
    if s.isdigit():
        return int(s)
    return ResultStatus.ERROR
```

Verify: tests assert both success and error branches.

```
python
def test_parse_int():
    assert parse_int("42") == 42
    assert parse_int("x") == ResultStatus.ERROR
```

10.2.3 Static Constants Module and Static Analysis Friendly Pattern

Intent: avoid magic numbers; centralize constants.

Code sketch

```
python
# constants.py
MAX_RETRIES = 5
TIMEOUT_MS = 2000
```

Verify: tests import constants and assert values; static analyzers can flag direct numeric literals elsewhere.

```
python
def test_constants():
```

```
import constants
assert constants.MAX_RETRIES == 5
```

10.2.4 Dependency Injection via Interfaces

Intent: inject behavior via explicit protocol, no reflection or dynamic imports.

Code sketch

```
python
from typing import Protocol

class Storage(Protocol):
    def save(self, key: str, value: str) → None: ...

def store_value(storage: Storage, key: str, value: str) → None:
    storage.save(key, value)
```

Verify: use a test double implementing save and assert it was called.

```
python
class Dummy:
    def __init__(self):
        self.calls = []

    def save(self, k, v):
        self.calls.append((k, v))

def test_store_value():
    d = Dummy()
    store_value(d, "k", "v")
    assert d.calls == [("k", "v")]
```

10.2.5 Mockable IO Pattern

Intent: make IO mockable and deterministic for tests.

Code sketch

```
python
def read_and_upper(read_fn):
    data = read_fn()
    return data.upper()
```

Verify: pass lambda returning fixed string and assert result.

```
python
def test_read_and_upper():
    assert read_and_upper(lambda: "abc") == "ABC"
```

10.3 Hard Examples (10)

10.3.1 Deterministic State Machine with Explicit Transitions

Intent: deterministic finite state machine, transitions explicit and symbolic.

Code sketch

```
python
from enum import Enum
from typing import Dict, Tuple

class State(Enum):
    IDLE = "idle"
    RUN = "run"
    STOP = "stop"

Transition = Dict[Tuple[State, str], State]

TRANSITIONS: Transition = {
    (State.IDLE, "start"): State.RUN,
    (State.RUN, "stop"): State.STOP,
}
```

```
def step(state: State, event: str) → State:
    return TRANSITIONS.get((state, event), state)
```

Verify: unit tests assert deterministic next states for fixed inputs.

python

```
def test_state_machine():
    assert step(State.IDLE, "start") == State.RUN
    assert step(State.RUN, "stop") == State.STOP
```

10.3.2 Deterministic Retry Logic Without Randomness

Intent: deterministic retry schedule using explicit backoff table.

Code sketch

python

```
BACKOFF_MS = [100, 200, 400]

def next_backoff(attempt: int) → int:
    return BACKOFF_MS[min(attempt, len(BACKOFF_MS)-1)]
```

Verify: tests assert exact backoff values for attempts.

python

```
def test_backoff():
    assert next_backoff(0) == 100
    assert next_backoff(5) == 400
```

10.3.3 Offline First Sync Simulator Symbolic

Intent: simulate deterministic sync steps without network randomness.

Code sketch

python

```
def sync_steps(local_changes: int) → int:
    # symbolic: each step applies one change
    return local_changes
```

Verify: tests assert mapping from input to output deterministically.

```
python
def test_sync_steps():
    assert sync_steps(3) == 3
```

10.3.4 Deterministic Concurrency via Explicit Context Passing

Intent: avoid global state; pass explicit context to worker functions.

Code sketch

```
python
from typing import Dict

def worker(ctx: Dict[str, int]) → int:
    return ctx["value"] + 1
```

Verify: tests pass different contexts and assert isolation.

```
python
def test_worker_context():
    assert worker({"value": 1}) == 2
    assert worker({"value": 5}) == 6
```

10.3.5 Feature Flags as Explicit Config, No Dynamic Imports

Intent: feature flags are explicit booleans in config.

Code sketch

```
python
def feature_enabled(cfg: dict, name: str) → bool:
    return bool(cfg.get("features", {}).get(name, False))
```

Verify: tests assert enabled/disabled deterministically.

```
python
def test_feature_enabled():
    cfg = {"features": {"x": True}}
    assert feature_enabled(cfg, "x") is True
```

```
assert feature_enabled(cfg, "y") is False
```

10.3.6 Deterministic Validation Pipeline

Intent: pipeline of pure validators returning explicit enums.

Code sketch

```
python
from enum import Enum
from typing import List

class Valid(Enum):
    OK = "ok"
    FAIL = "fail"

def validate_nonempty(s: str) → Valid:
    return Valid.OK if s else Valid.FAIL

def run_validators(s: str, validators: List) → Valid:
    for v in validators:
        if v(s) == Valid.FAIL:
            return Valid.FAIL
    return Valid.OK
```

Verify: tests assert pipeline outcomes for fixed inputs.

```
python
def test_validators():
    assert run_validators("x", [validate_nonempty]) == Valid.OK
    assert run_validators("", [validate_nonempty]) == Valid.FAIL
```

10.3.7 Deterministic Serialization Schema Example

Intent: explicit schema mapping, no reflection.

Code sketch

```
python
def serialize_user(u: dict) → dict:
    return {"id": u["id"], "name": u["name"]}
```

Verify: tests assert exact serialized shape.

```
python
def test_serialize_user():
    assert serialize_user({"id":1,"name":"a","extra":True}) == {"id":1,"name":"a"}
```

10.3.8 Safe Concurrency Pattern with Explicit Locks Passed In

Intent: pass lock object explicitly; avoid global locks.

Code sketch

```
python
from threading import Lock

def increment(shared: dict, key: str, lock: Lock):
    with lock:
        shared[key] = shared.get(key, 0) + 1
```

Verify: tests use a lock and deterministic sequence to assert final value.

```
python
def test_increment():
    lock = Lock()
    shared = {}
    increment(shared, "c", lock)
```

```
assert shared["c"] == 1
```

10.3.9 Deterministic Metrics Collector with Explicit Flush

Intent: collect metrics in memory and flush deterministically.

Code sketch

```
python
def record_metric(store: dict, name: str, value: int):
    store.setdefault(name, 0)
    store[name] += value

def flush_metrics(store: dict) → dict:
    out = dict(store)
    store.clear()
    return out
```

Verify: tests assert recorded values and flush behavior.

```
python
def test_metrics():
    s = {}
    record_metric(s, "m", 1)
    assert flush_metrics(s) == {"m":1}
    assert s == {}
```


10.3.10 Plugin Pattern Without Dynamic Imports

Intent: explicit plugin registry; no runtime imports or reflection.

Code sketch

python

```
PLUGINS = {}
```

```
def register(name: str, fn):
```

```
    PLUGINS[name] = fn
```

```
def run_plugin(name: str, *args, **kwargs):
```

```
    return PLUGINS[name](*args, **kwargs)
```

Verify: tests register a dummy plugin and assert deterministic output.

python

```
def test_plugins():
```

```
    register("p", lambda x: x+1)
```

```
    assert run_plugin("p", 1) == 2
```

10.4 Verification Patterns and Testing Guidance

General VNAGY testing principles used across examples

- **Unit tests assert exact outputs** for fixed inputs; avoid probabilistic assertions.
- **Prefer enums and typed results** over magic values or exceptions for control flow.
- **Use dependency injection** to pass IO, locks, and external services so tests can supply deterministic doubles.
- **Avoid global state in tests;** create fresh instances per test to ensure isolation.
- **Use `io.StringIO` or simple callables** to simulate IO deterministically.

- **Static analysis readiness:** centralize numeric constants and avoid inline literals so static analyzers can flag violations.
- **Test checklist for each example:**
 - deterministic input → deterministic output
 - no implicit IO used in test
 - enums used where applicable
 - no dynamic imports or runtime code generation in test scope
 - tests are pure assertions, not timing or race-dependent

10.5 Appendix A Checklist

- [] 5 easy examples included and unit-testable.
- [] 5 medium examples included and unit-testable.
- [] 10 hard examples included and unit-testable.
- [] All examples follow **VNAGY** constraints: symbolic, deterministic, explicit.
- [] Each example has a clear verification pattern (unit test sketch).
- [] No dynamic imports, runtime code generation, reflection, randomness, or hidden IO in examples.
- [] Enums and typed constructs used where appropriate.
- [] Constants centralized for static analysis friendliness.

11. Appendix B — Rust Examples

11.1 Easy Examples (5)

12.1.1 Pure Function Transformation

Intent: deterministic, side-effect free transformation.

Code sketch

```
rust
pub fn normalize_tags(tags: &[&str]) → Vec<String> {
    tags.iter().map(|t| t.trim().to_lowercase()).collect()
}
```

Verify: unit test asserts exact output for fixed input.

```
rust
#[test]
fn normalize_tags_basic() {
    assert_eq!(normalize_tags(&[" A ", "B "]), vec!["a".to_string(), "b".to_string()]);
}
```

11.1.2 Enums for Signals and Errors

Intent: use enums instead of magic numbers/strings.

Code sketch

```
rust
#[derive(Debug, PartialEq)]
pub enum Signal { Start, Stop }

#[derive(Debug, PartialEq)]
pub enum ErrorCode { Invalid = 1, Timeout = 2 }
```

Verify: tests assert enum identity and values.

```
rust
```

```
#[test]
fn enums_values() {
    assert_eq!(Signal::Start, Signal::Start);
    assert_eq!(ErrorCode::Invalid as isize, 1);
}
```

11.1.3 Explicit IO via Parameters

Intent: avoid implicit IO; pass readers explicitly.

Code sketch

```
rust
use std::io::{self, Read};

pub fn count_lines<R: Read>(mut reader: R) → io::Result<usize> {
    let mut s = String::new();
    reader.read_to_string(&mut s)?;
    Ok(s.lines().count())
}
```

Verify: use Cursor in tests to assert deterministic count.

```
rust
#[test]
fn count_lines_deterministic() {
    use std::io::Cursor;
    let c = Cursor::new("a\nb\n");
    assert_eq!(count_lines(c).unwrap(), 2);
}
```

11.1.4 No Global State (Factory Pattern)

Intent: demonstrate isolated state via explicit instances.

Code sketch

```
rust
pub struct Counter { count: usize }

impl Counter {
    pub fn new() → Self { Self { count: 0 } }
    pub fn inc(&mut self) → usize { self.count += 1; self.count }
}
```

Verify: create two counters and assert isolation.

```
rust
#[test]
fn counters_isolated() {
    let mut a = Counter::new();
    let mut b = Counter::new();
    assert_eq!(a.inc(), 1);
    assert_eq!(b.inc(), 1);
}
```

11.1.5 Deterministic Formatter

Intent: deterministic formatting with explicit parameters.

Code sketch

```
rust
pub fn format_amount(amount: i64, currency: &str) → String {
    format!("{}", amount, currency)
}
```

Verify: assert exact string for given inputs.

```
rust
#[test]
fn format_amount_exact() {
    assert_eq!(format_amount(100, "USD"), "100 USD");
}
```

11.2.1 Explicit Config Struct (No Dynamic Loading)

Intent: deterministic behavior driven by an explicit config struct.

Code sketch

```
rust
#[derive(Debug)]
pub struct Config { pub retries: u8, pub timeout_ms: u32 }

pub fn perform_task(cfg: &Config, payload: &str) → String {
    format!("processed:{}:{}", payload, cfg.retries)
}
```

Verify: instantiate Config and assert deterministic output.

```
rust
#[test]
fn perform_task_deterministic() {
    let cfg = Config { retries: 3, timeout_ms: 1000 };
    assert_eq!(perform_task(&cfg, "x"), "processed:x:3");
}
```

11.2 Medium Examples (5)

11.2.1 Explicit Config Struct (No Dynamic Loading)

Intent: deterministic behavior driven by an explicit config struct.

Code sketch

```
rust
#[derive(Debug)]
pub struct Config { pub retries: u8, pub timeout_ms: u32 }

pub fn perform_task(cfg: &Config, payload: &str) → String {
    format!("processed:{}:{}", payload, cfg.retries)
}
```

Verify: instantiate Config and assert deterministic output.

```
#[test]
fn perform_task_deterministic() {
    let cfg = Config { retries: 3, timeout_ms: 1000 };
    assert_eq!(perform_task(&cfg, "x"), "processed:x:3");
}
```

11.2.2 Explicit Error Handling with Result and Enum

Intent: typed results; avoid exceptions for control flow.

Code sketch

```
rust
#[derive(Debug, PartialEq)]
pub enum ParseError { Invalid }

pub fn parse_int(s: &str) -> Result<i64, ParseError> {
    s.parse::<i64>().map_err(|_| ParseError::Invalid)
}
```

Verify: tests assert both success and error branches.

```
rust
#[test]
fn parse_int_branches() {
    assert_eq!(parse_int("42"), Ok(42));
    assert_eq!(parse_int("x"), Err(ParseError::Invalid));
}
```

11.2.3 Centralized Constants for Static Analysis

Intent: avoid magic numbers; centralize constants for static checks.

Code sketch

```
rust
pub mod constants {
    pub const MAX_RETRIES: u8 = 5;
    pub const TIMEOUT_MS: u32 = 2000;
}
```

Verify: tests import constants and assert values.

```
rust
#[test]
fn constants_values() {
    assert_eq!(constants::MAX_RETRIES, 5);
}
```

11.2.4 Trait-based Dependency Injection (No Reflection)

Intent: inject behavior via explicit trait; test doubles replace implementations.

Code sketch

```
rust
pub trait Storage { fn save(&mut self, key: &str, value: &str); }

pub fn store_value<S: Storage>(storage: &mut S, key: &str, value: &str) {
```



```

    storage.save(key, value);
}

```

Verify: implement a dummy Storage in tests and assert recorded calls.

```

rust
#[test]
fn store_value_calls() {
    struct Dummy { calls: Vec<(String,String)> }
    impl Dummy { fn new() → Self { Self { calls: vec![] } } }
    impl Storage for Dummy { fn save(&mut self, k: &str, v: &str)
{ self.calls.push((k.to_string(), v.to_string())); } }
    let mut d = Dummy::new();
    store_value(&mut d, "k", "v");
    assert_eq!(d.calls, vec![("k".to_string(), "v".to_string())]);
}

```

11.2.5 Mockable IO via Closures

Intent: make IO deterministic and testable by passing closures.

Code sketch

```

rust
pub fn read_and_upper<F: Fn() → String>(read_fn: F) → String {
    read_fn().to_uppercase()
}

```

Verify: pass closure returning fixed string and assert result.

```

rust
#[test]
fn read_and_upper_deterministic() {
    assert_eq!(read_and_upper(|| "abc".to_string()), "ABC");
}

```

11.3 Hard Examples (5)

11.3.1 Deterministic State Machine (Explicit Transitions)

Intent: finite state machine with explicit transition mapping.

Code sketch

```
rust
#[derive(Debug, PartialEq, Eq, Hash, Clone, Copy)]
pub enum State { Idle, Run, Stop }

pub fn step(state: State, event: &str) → State {
    match (state, event) {
        (State::Idle, "start") ⇒ State::Run,
        (State::Run, "stop") ⇒ State::Stop,
        (s, _) ⇒ s,
    }
}
```

Verify: unit tests assert deterministic next states for fixed inputs.

```
rust
#[test]
fn state_machine_transitions() {
    assert_eq!(step(State::Idle, "start"), State::Run);
    assert_eq!(step(State::Run, "stop"), State::Stop);
}
```

11.3.2 Deterministic Retry Schedule (Explicit Backoff Table)

Intent: deterministic retry schedule using an explicit table; no randomness. **Code sketch**

```
rust
pub const BACKOFF_MS: [u64; 3] = [100, 200, 400];
```

```
pub fn next_backoff(attempt: usize) → u64 {
    BACKOFF_MS[attempt.min(BACKOFF_MS.len() - 1)]
}
```

Verify: tests assert exact backoff values for attempts.

```
rust
#[test]
fn backoff_values() {
    assert_eq!(next_backoff(0), 100);
    assert_eq!(next_backoff(5), 400);
}
```

11.3.3 Deterministic Validation Pipeline (Pure Validators)

Intent: pipeline of pure validators returning explicit enum results.

Code sketch

```
rust
#[derive(Debug, PartialEq)]
pub enum Valid { Ok, Fail }

pub fn validate_nonempty(s: &str) → Valid {
    if s.is_empty() { Valid::Fail } else { Valid::Ok }
}

pub fn run_validators(s: &str, validators: &[fn(&str) → Valid]) → Valid {
    for v in validators {
```

```

        if v(s) == Valid::Fail { return Valid::Fail; }
    }
    Valid::Ok
}

```

Verify: tests assert pipeline outcomes for fixed inputs.

```

rust
#[test]
fn validators_pipeline() {
    assert_eq!(run_validators("x", &[validate_nonempty]), Valid::Ok);
    assert_eq!(run_validators("", &[validate_nonempty]), Valid::Fail);
}

```

11.3.4 Deterministic Metrics Collector with Explicit Flush

Intent: collect metrics in memory and flush deterministically; no hidden IO.

Code sketch

```

rust
use std::collections::HashMap;

pub fn record_metric(store: &mut HashMap<String, i64>, name: &str, value: i64) {
    *store.entry(name.to_string()).or_insert(0) += value;
}

pub fn flush_metrics(store: &mut HashMap<String, i64>) → HashMap<String, i64> {
    let out = store.clone();
    store.clear();
    out
}

```

Verify: tests assert recorded values and flush behavior.

```
rust
```

```
#[test]
fn metrics_record_and_flush() {
    let mut s = HashMap::new();
    record_metric(&mut s, "m", 1);
    assert_eq!(flush_metrics(&mut s).get("m"), Some(&1));
    assert!(s.is_empty());
}
```

11.3.5 Plugin Registry Without Dynamic Loading

Intent: explicit plugin registry; no runtime imports or reflection.

Code sketch

```
rust
use std::collections::HashMap;
pub type PluginFn = fn(i32) → i32;

pub struct Registry { plugins: HashMap<String, PluginFn> }

impl Registry {
    pub fn new() → Self { Self { plugins: HashMap::new() } }
    pub fn register(&mut self, name: &str, f: PluginFn) { self.plugins.insert(name.to_string(), f); }
    pub fn run(&self, name: &str, arg: i32) → Option<i32> { self.plugins.get(name).map(|f| f(arg)) }
}
```

Verify: tests register a dummy plugin and assert deterministic output.

```
rust
#[test]
fn plugin_registry_deterministic() {
    let mut r = Registry::new();
```

```

r.register("p", |x| x + 1);
assert_eq!(r.run("p", 1), Some(2));
}

```

11.4 Verification Patterns and Testing Guidance

- **Unit tests assert exact outputs** for fixed inputs; avoid probabilistic assertions.
- **Prefer enums and typed results** over magic values or exceptions for control flow.
- **Use dependency injection** to pass IO, locks, and external services so tests can supply deterministic doubles.
- **Avoid global mutable state**; create fresh instances per test to ensure isolation.
- **Simulate IO deterministically** with Cursor or closures.
- **Centralize numeric constants** to support static analysis and detect magic numbers.
- **Test checklist for each example:** deterministic input → deterministic output; no implicit IO; enums where applicable; no dynamic loading or runtime code generation; tests are pure assertions.

11.5 Appendix B Checklist

- [] 5 easy examples included and unit-testable.
- [] 5 medium examples included and unit-testable.
- [] 5 hard examples included and unit-testable.
- [] All examples follow **VNAGY** constraints: symbolic, deterministic, explicit.
- [] Each example has a clear verification pattern (unit test sketch).
- [] No dynamic imports, runtime code generation, reflection, randomness, or hidden IO in examples.
- [] Enums and typed constructs used where appropriate.
- [] Constants centralized for static analysis friendliness.

12. Appendix C — Combined Patterns

Hardened Example 1 — Deterministic Message Envelope (HMAC + size limits)

Intent Language-agnostic envelope with authenticated integrity and strict size limits.

Security measures

- HMAC-SHA256 for integrity (requires secret key).
- MAX_PAYLOAD_LEN enforced before allocation.
- Deterministic big-endian layout.
- Typed errors (Result / exceptions).
- No implicit IO; functions operate on byte slices.

Rust (sketch)

```
use hmac::{Hmac, Mac};
use sha2::Sha256;
use std::convert::TryInto;

type HmacSha256 = Hmac<Sha256>;
const MAX_PAYLOAD_LEN: usize = 1_048_576; // 1 MiB

#[derive(Debug)]
pub enum EnvelopeError { Truncated, Oversize, BadHmac, ParseError }

pub struct Envelope {
    pub version: u8,
    pub schema_id: u16,
```

```

    pub payload_type: u8,
    pub payload: Vec<u8>,
}

```

```

impl Envelope {

```

```

    pub fn to_bytes(&self, hmac_key: &[u8]) → Vec<u8> {
        let mut out = Vec::with_capacity(1 + 2 + 1 + 4 + self.payload.len() + 32);
        out.push(self.version);
        out.extend(&self.schema_id.to_be_bytes());
        out.push(self.payload_type);
        out.extend(&(self.payload.len() as u32).to_be_bytes());
        out.extend(&self.payload);
        let mut mac = HmacSha256::new_from_slice(hmac_key).expect("invalid key");
        mac.update(&out);
        out.extend(&mac.finalize().into_bytes());
        out
    }
}

```

```

    pub fn from_bytes(data: &[u8], hmac_key: &[u8]) → Result<Self, EnvelopeError> {
        if data.len() < 1 + 2 + 1 + 4 + 32 { return Err(EnvelopeError::Truncated); }
        let hmac_offset = data.len() - 32;
        // verify HMAC before allocating payload
        let mut mac = HmacSha256::new_from_slice(hmac_key).map_err(|_|
EnvelopeError::ParseError?);
        mac.update(&data[..hmac_offset]);
        mac.verify_slice(&data[hmac_offset..]).map_err(|_| EnvelopeError::BadHmac?);
        let version = data[0];
        let schema_id = u16::from_be_bytes(data[1..3].try_into()).map_err(|_|
EnvelopeError::ParseError?);

```



```

    let payload_type = data[3];

    let len = u32::from_be_bytes(data[4..8].try_into().map_err(|_|
EnvelopeError::ParseError?)) as usize;

    if len > MAX_PAYLOAD_LEN { return Err(EnvelopeError::Oversize); }

    if data.len() < 8 + len + 32 { return Err(EnvelopeError::Truncated); }

    let payload = data[8..8+len].to_vec();

    Ok(Envelope { version, schema_id, payload_type, payload })
}
}

```

Python (sketch)

```

import hmac, hashlib

MAX_PAYLOAD_LEN = 1_048_576 # 1 MiB

class EnvelopeError(Exception): pass

def to_bytes(envelope: dict, hmac_key: bytes) → bytes:
    out = bytearray()
    out.append(envelope["version"])
    out.extend(envelope["schema_id"].to_bytes(2, "big"))
    out.append(envelope["payload_type"])
    out.extend(len(envelope["payload"]).to_bytes(4, "big"))
    out.extend(envelope["payload"])
    mac = hmac.new(hmac_key, out, hashlib.sha256).digest()
    out.extend(mac)
    return bytes(out)

def from_bytes(data: bytes, hmac_key: bytes) → dict:
    if len(data) < 1+2+1+4+32:

```

```

        raise EnvelopeError("truncated")
    body, mac = data[:-32], data[-32:]
    if not hmac.compare_digest(hmac.new(hmac_key, body, hashlib.sha256).digest(), mac):
        raise EnvelopeError("bad_hmac")
    version = body[0]
    schema_id = int.from_bytes(body[1:3], "big")
    payload_type = body[3]
    length = int.from_bytes(body[4:8], "big")
    if length > MAX_PAYLOAD_LEN:
        raise EnvelopeError("oversize")
    if len(body) < 8 + length:
        raise EnvelopeError("truncated")
    payload = body[8:8+length]
    return {"version": version, "schema_id": schema_id, "payload_type": payload_type,
            "payload": payload}

```

Verification

- Roundtrip: Rust to_bytes → Python from_bytes → assert fields and payload equal.
- Tamper: flip a byte in serialized data → from_bytes raises bad_hmac.
- Oversize: craft header with len = MAX_PAYLOAD_LEN + 1 → from_bytes raises oversize.
- Fuzz: run fuzzers against from_bytes in both languages; assert no crashes and only typed errors.

Hardened Example 2 — Safe Structured Deserialization with Schema Gate

Intent Validate message content deterministically against a minimal, versioned schema registry before processing.

Security measures

- Whitelisted schema_id values.

- Field length limits (e.g., MAX_STRING_LEN).
- Deterministic validator returning explicit error codes.
- No exceptions used for control flow in Rust; explicit Result.

Rust (sketch)

```
use serde_json::Value;

pub enum ValidateError { MissingField(&'static str), WrongType(&'static str),
Oversize(&'static str), UnknownSchema }

const MAX_STRING_LEN: usize = 1024;
const ALLOWED_SCHEMAS: &[u16] = &[1, 2];

pub fn validate(schema_id: u16, v: &Value) → Result<(), ValidateError> {
    if !ALLOWED_SCHEMAS.contains(&schema_id) { return
Err(ValidateError::UnknownSchema); }
    if schema_id == 1 {
        let id = v.get("id").ok_or(ValidateError::MissingField("id"))?;
        if !id.is_string() { return Err(ValidateError::WrongType("id")); }
        if id.as_str().unwrap().len() > MAX_STRING_LEN { return
Err(ValidateError::Oversize("id")); }
        let name = v.get("name").ok_or(ValidateError::MissingField("name"))?;
        if !name.is_string() { return Err(ValidateError::WrongType("name")); }
        if name.as_str().unwrap().len() > MAX_STRING_LEN { return
Err(ValidateError::Oversize("name")); }
    }
    Ok(())
}
```

Python (sketch)

```
python
MAX_STRING_LEN = 1024
```

```
ALLOWED_SCHEMAS = {1, 2}
```

```
class ValidateError(Exception): pass
```

```
def validate(schema_id: int, obj: dict):  
    if schema_id not in ALLOWED_SCHEMAS:  
        raise ValidateError("unknown_schema")  
    if schema_id == 1:  
        if "id" not in obj:  
            raise ValidateError("missing:id")  
        if not isinstance(obj["id"], str):  
            raise ValidateError("wrongtype:id")  
        if len(obj["id"]) > MAX_STRING_LEN:  
            raise ValidateError("oversize:id")  
        if "name" not in obj:  
            raise ValidateError("missing:name")  
        if not isinstance(obj["name"], str):  
            raise ValidateError("wrongtype:name")  
        if len(obj["name"]) > MAX_STRING_LEN:  
            raise ValidateError("oversize:name")  
    return True
```

Verification

- Vector tests: run identical test vectors (valid, missing, wrong type, oversize) in Rust and Python; assert identical error categories.
- Unknown schema: message with schema_id not in whitelist returns UnknownSchema.
- Fuzzing: fuzz JSON inputs; ensure validator never panics and always returns typed errors.

Hardened Example 3 — Secure File Exchange and Atomic Persistence (framing + file lock + atomic write)

Intent Append framed messages to disk safely and read them deterministically; protect against partial writes, oversize frames, and concurrent writers.

Security measures

- Framing: 4-byte big-endian length + payload + HMAC.
- MAX_FRAME enforced before reading payload.
- File locking for single-writer guarantee.
- Atomic write for snapshots (temp file + rename) and fsync.
- Typed IO errors surfaced to caller.

Rust (sketch)

```
use std::fs::{OpenOptions, File};
use std::io::{Read, Write, BufReader};
use fs2::FileExt; // crate fs2 for file locking
use hmac::{Hmac, Mac};
use sha2::Sha256;

type HmacSha256 = Hmac<Sha256>;
const MAX_FRAME: usize = 2_097_152; // 2 MiB

pub fn append_frame(path: &str, payload: &[u8], hmac_key: &[u8]) → std::io::Result<()> {
    if payload.len() > MAX_FRAME {
        return Err(std::io::Error::new(std::io::ErrorKind::InvalidInput, "oversize"));
    }

    let mut f = OpenOptions::new().create(true).append(true).open(path)?;
    f.lock_exclusive()?; // block until exclusive lock acquired

    // build frame in memory (bounded)
    let len_bytes = (payload.len() as u32).to_be_bytes();
```

```

let mut frame = Vec::with_capacity(4 + payload.len() + 32);
frame.extend(&len_bytes);
frame.extend(payload);

let mut mac = HmacSha256::new_from_slice(hmac_key).map_err(|_|
std::io::Error::new(std::io::ErrorKind::Other, "hmac"))?;

mac.update(&frame);
frame.extend(&mac.finalize().into_bytes());

// write and sync
f.write_all(&frame)?;
f.sync_all()?;
f.unlock()?;
Ok(())
}

pub fn read_frames(path: &str, hmac_key: &[u8]) → std::io::Result<Vec<Vec<u8>>> {
let f = File::open(path)?;
let mut reader = BufReader::new(f);
let mut frames = Vec::new();
loop {
let mut len_buf = [0u8; 4];
if let Err(e) = reader.read_exact(&mut len_buf) {
if e.kind() == std::io::ErrorKind::UnexpectedEof { break; } else { return Err(e); }
}

let len = u32::from_be_bytes(len_buf) as usize;

if len > MAX_FRAME { return Err(std::io::Error::new(std::io::ErrorKind::InvalidData,
"oversize")); }

let mut payload = vec![0u8; len];
reader.read_exact(&mut payload)?;

let mut mac = [0u8; 32];

```

```

    reader.read_exact(&mut mac)?;

    // verify HMAC

    let mut mac_calc = HmacSha256::new_from_slice(hmac_key).map_err(|_|
std::io::Error::new(std::io::ErrorKind::Other, "hmac"))?;

    let mut frame_body = Vec::with_capacity(4 + len);

    frame_body.extend(&len_buf);
    frame_body.extend(&payload);
    mac_calc.update(&frame_body);

    mac_calc.verify_slice(&mac).map_err(|_|
std::io::Error::new(std::io::ErrorKind::InvalidData, "bad_hmac"))?;

    frames.push(payload);
}
Ok(frames)
}

```

Python (sketch)

```

import os, hmac, hashlib, struct, fcntl

MAX_FRAME = 2_097_152 # 2 MiB

def append_frame(path: str, payload: bytes, hmac_key: bytes):
    if len(payload) > MAX_FRAME:
        raise ValueError("oversize")
    with open(path, "ab") as f:
        # acquire exclusive lock
        fcntl.flock(f.fileno(), fcntl.LOCK_EX)
        frame = struct.pack(">I", len(payload)) + payload
        mac = hmac.new(hmac_key, frame, hashlib.sha256).digest()
        f.write(frame + mac)
    f.flush()

```

```

os.fsync(f.fileno())

fcntl.flock(f.fileno(), fcntl.LOCK_UN)

def read_frames(path: str, hmac_key: bytes):
    frames = []
    with open(path, "rb") as f:
        while True:
            len_bytes = f.read(4)
            if not len_bytes:
                break
            if len(len_bytes) < 4:
                raise IOError("truncated")
            length = struct.unpack(">I", len_bytes)[0]
            if length > MAX_FRAME:
                raise ValueError("oversize")
            payload = f.read(length)
            if len(payload) < length:
                raise IOError("truncated")
            mac = f.read(32)
            if len(mac) < 32:
                raise IOError("truncated")
            if not hmac.compare_digest(hmac.new(hmac_key, len_bytes + payload,
hashlib.sha256).digest(), mac):
                raise IOError("bad_hmac")
            frames.append(payload)
    return frames

```

Verification

- Append/read roundtrip: append frames in Rust, read in Python, assert payload equality.

- Oversize: attempt to append a payload > MAX_FRAME → writer rejects with oversize.
- Partial write resilience: simulate truncated file (cut last N bytes) → reader returns truncated error, not panic.
- Concurrency: run multiple appenders; file lock ensures no interleaving; verify all frames present and intact.
- CI: include tamper test (modify a frame's byte) → reader reports bad_hmac.

Final verification checklist (apply to all examples)

- Implement and store HMAC keys securely (not hardcoded).
- Add unit tests for normal, oversize, truncated, and tampered inputs.
- Add cross-language roundtrip tests in CI.
- Run fuzzing on deserializers.
- Enforce limits (MAX_PAYLOAD_LEN, MAX_FRAME, MAX_STRING_LEN) and assert in tests.
- Ensure file locks and atomic writes are used where concurrent writers or persistence exist.

Hardened Example 3 — Secure File Exchange and Atomic Persistence (framing + file lock + atomic write)

Intent Append framed messages to disk safely and read them deterministically; protect against partial writes, oversize frames, and concurrent writers.

Security measures

- Framing: 4-byte big-endian length + payload + HMAC.
- MAX_FRAME enforced before reading payload.
- File locking for single-writer guarantee.
- Atomic write for snapshots (temp file + rename) and fsync.
- Typed IO errors surfaced to caller.

Rust (sketch)

```
use std::fs::{OpenOptions, File};
use std::io::{Read, Write, BufReader};
use fs2::FileExt; // crate fs2 for file locking
use hmac::{Hmac, Mac};
use sha2::Sha256;

type HmacSha256 = Hmac<Sha256>;
const MAX_FRAME: usize = 2_097_152; // 2 MiB

pub fn append_frame(path: &str, payload: &[u8], hmac_key: &[u8]) → std::io::Result<> {
    if payload.len() > MAX_FRAME {
        return Err(std::io::Error::new(std::io::ErrorKind::InvalidInput, "oversize"));
    }

    let mut f = OpenOptions::new().create(true).append(true).open(path)?;
    f.lock_exclusive()?; // block until exclusive lock acquired
    // build frame in memory (bounded)

    let len_bytes = (payload.len() as u32).to_be_bytes();
```

```

let mut frame = Vec::with_capacity(4 + payload.len() + 32);

frame.extend(&len_bytes);

frame.extend(payload);

        let mut mac = HmacSha256::new_from_slice(hmac_key).map_err(|_|
std::io::Error::new(std::io::ErrorKind::Other, "hmac"))?;
    mac.update(&frame);
    frame.extend(&mac.finalize().into_bytes());
    // write and sync
    f.write_all(&frame)?;
    f.sync_all()?;
    f.unlock()?;
    Ok(())
}

pub fn read_frames(path: &str, hmac_key: &[u8]) → std::io::Result<Vec<Vec<u8>>> {
    let f = File::open(path)?;
    let mut reader = BufReader::new(f);
    let mut frames = Vec::new();
    loop {
        let mut len_buf = [0u8; 4];
        if let Err(e) = reader.read_exact(&mut len_buf) {
            if e.kind() == std::io::ErrorKind::UnexpectedEof { break; } else { return Err(e); }
        }
        let len = u32::from_be_bytes(len_buf) as usize;
        if len > MAX_FRAME { return Err(std::io::Error::new(std::io::ErrorKind::InvalidData,
"oversize")); }
        let mut payload = vec![0u8; len];
        reader.read_exact(&mut payload)?;
        let mut mac = [0u8; 32];

```

```

    reader.read_exact(&mut mac)?;

    // verify HMAC

    let mut mac_calc = HmacSha256::new_from_slice(hmac_key).map_err(|_|
std::io::Error::new(std::io::ErrorKind::Other, "hmac"))?;

    let mut frame_body = Vec::with_capacity(4 + len);

    frame_body.extend(&len_buf);
    frame_body.extend(&payload);
    mac_calc.update(&frame_body);

    mac_calc.verify_slice(&mac).map_err(|_|
std::io::Error::new(std::io::ErrorKind::InvalidData, "bad_hmac"))?;

    frames.push(payload);
}
Ok(frames)
}

```

Python (sketch)

```

import os, hmac, hashlib, struct, fcntl

MAX_FRAME = 2_097_152 # 2 MiB

def append_frame(path: str, payload: bytes, hmac_key: bytes):
    if len(payload) > MAX_FRAME:
        raise ValueError("oversize")

    with open(path, "ab") as f:
        # acquire exclusive lock
        fcntl.flock(f.fileno(), fcntl.LOCK_EX)

        frame = struct.pack(">I", len(payload)) + payload
        mac = hmac.new(hmac_key, frame, hashlib.sha256).digest()

        f.write(frame + mac)

    f.flush()

```

```

os.fsync(f.fileno())

fcntl.flock(f.fileno(), fcntl.LOCK_UN)

def read_frames(path: str, hmac_key: bytes):
    frames = []
    with open(path, "rb") as f:
        while True:
            len_bytes = f.read(4)
            if not len_bytes:
                break
            if len(len_bytes) < 4:
                raise IOError("truncated")
            length = struct.unpack(">I", len_bytes)[0]
            if length > MAX_FRAME:
                raise ValueError("oversize")
            payload = f.read(length)
            if len(payload) < length:
                raise IOError("truncated")
            mac = f.read(32)
            if len(mac) < 32:
                raise IOError("truncated")
            if not hmac.compare_digest(hmac.new(hmac_key, len_bytes + payload,
hashlib.sha256).digest(), mac):
                raise IOError("bad_hmac")
            frames.append(payload)
    return frames

```

Hardened Example 4 Rust completion and verification

Rust `append_frame` and `read_frames` completed

```
use std::fs::{OpenOptions, File};
use std::io::{Read, Write, BufReader};
use fs2::FileExt; // crate fs2 for file locking
use hmac::{Hmac, Mac};
use sha2::Sha256;
use std::path::Path;

type HmacSha256 = Hmac<Sha256>;
const MAX_FRAME: usize = 2_097_152; // 2 MiB

pub fn append_frame(path: &str, payload: &[u8], hmac_key: &[u8]) → std::io::Result<()> {
    if payload.len() > MAX_FRAME {
        return Err(std::io::Error::new(std::io::ErrorKind::InvalidInput, "oversize"));
    }
    let mut f = OpenOptions::new().create(true).append(true).open(path)?;
    // Acquire exclusive lock for single-writer guarantee
    f.lock_exclusive()?;
    // Build bounded frame: 4-byte length + payload + 32-byte HMAC
    let len_bytes = (payload.len() as u32).to_be_bytes();
    let mut frame = Vec::with_capacity(4 + payload.len() + 32);
    frame.extend(&len_bytes);
    frame.extend(payload);
    let mut mac = HmacSha256::new_from_slice(hmac_key)
        .map_err(|_| std::io::Error::new(std::io::ErrorKind::Other, "invalid hmac key"))?;
    mac.update(&frame);
    frame.extend(&mac.finalize().into_bytes());
    // Write atomically with sync
```

```

    f.write_all(&frame)?;
    f.sync_all()?;
    // Release lock
    f.unlock()?;
    Ok(())
}

pub fn read_frames(path: &str, hmac_key: &[u8]) → std::io::Result<Vec<Vec<u8>>> {
    if !Path::new(path).exists() {
        return Ok(Vec::new());
    }
    let f = File::open(path)?;
    let mut reader = BufReader::new(f);
    let mut frames = Vec::new();
    loop {
        let mut len_buf = [0u8; 4];
        match reader.read_exact(&mut len_buf) {
            Ok(_) ⇒ {},
            Err(e) if e.kind() == std::io::ErrorKind::UnexpectedEof ⇒ break,
            Err(e) ⇒ return Err(e),
        }
        let len = u32::from_be_bytes(len_buf) as usize;
        if len > MAX_FRAME {
            return Err(std::io::Error::new(std::io::ErrorKind::InvalidData, "oversize"));
        }
        let mut payload = vec![0u8; len];
        reader.read_exact(&mut payload)?;
        let mut mac = [0u8; 32];
    }
}

```

```

    reader.read_exact(&mut mac)?;
    // Verify HMAC before accepting payload
    let mut mac_calc = HmacSha256::new_from_slice(hmac_key)
        .map_err(|_| std::io::Error::new(std::io::ErrorKind::Other, "invalid hmac key"))?;
    let mut frame_body = Vec::with_capacity(4 + len);
    frame_body.extend(&len_buf);
    frame_body.extend(&payload);
    mac_calc.update(&frame_body);
    mac_calc.verify_slice(&mac)
        .map_err(|_| std::io::Error::new(std::io::ErrorKind::InvalidData, "bad_hmac"))?;
    frames.push(payload);
}
Ok(frames)
}

```

Security notes applied in this code

- **HMAC SHA256** verifies integrity and detects tampering.
- **MAX_FRAME** prevents large allocations and allocation attacks.
- **File locking** (`fs2::FileExt::lock_exclusive`) enforces single-writer semantics.
- **Atomic write semantics** for frames achieved by writing complete framed record and calling `sync_all` before unlocking.
- **Typed IO errors** returned to caller for deterministic handling.
- **No unsafe** and no dynamic code execution.

Verification steps to add to CI

- **Roundtrip test:** append frames with Rust `append_frame`, read with Python `read_frames`, assert payload equality.
- **Oversize rejection:** attempt to append payload > `MAX_FRAME` and assert writer returns `oversize` error.
- **Tamper detection:** append a valid frame, flip one byte in file, run `read_frames` and assert `bad_hmac` error for the tampered frame.

- **Partial write resilience:** truncate file mid-frame and assert reader returns truncated/UnexpectedEof rather than panicking.
- **Concurrency test:** run multiple appenders in parallel; assert no interleaved frames and all frames present and intact.
- **Fuzzing:** fuzz read_frames with random/truncated inputs and assert no panics and only typed errors

Quick checklist before production

- Store HMAC keys securely (not in source).
- Lock file paths to an isolated directory with least privileges.
- Add monitoring for file size and rotate/compact logs deterministically.
- Add cross-language tests (Rust ↔ Python) to CI and run fuzzers on deserializers.

13. Revision History — Concise Documentation Section

Version Table

A structured record of all released versions of the document. Each entry includes:

- Version number
- Release date
- Author or responsible team
- Summary of changes
- Status (Draft, Review, Final)

This table ensures traceability and allows developers to understand how the specification evolved over time.

Structural Changes

This section lists modifications to the **architecture, layout, or organization** of the document or system. Examples include:

- Reorganized chapters or sections
- Introduced new conceptual blocks
- Updated diagrams or data flows
- Added or removed modules in the structural overview

Structural changes help readers quickly identify shifts in how the system is presented or conceptualized.

Content Additions

A summary of **newly added technical content**, such as:

- New definitions, rules, or constraints
- Additional security requirements
- Expanded examples or code patterns

- New validation logic or protocol details
- Clarifications based on developer feedback

This section highlights what is *newly introduced*, making it easy for developers to focus on updates without re-reading the entire document.

14. Glossary

Deterministic Execution

A system behavior where the same inputs always produce the same outputs, regardless of timing, environment, or concurrency. Deterministic execution eliminates hidden variability and makes validation, debugging, and security analysis predictable.

Explicit State

All system state is clearly defined, visible, and intentionally managed. No hidden globals, implicit caches, or side-effects. Explicit state ensures traceability, reproducibility, and secure reasoning about system behavior.

Minimal Surface

The principle of reducing the number of exposed interfaces, features, and interactions to the smallest necessary set. A minimal surface lowers attack vectors, simplifies auditing, and improves long-term maintainability.

Offline-First

A design approach where the system must operate correctly without network connectivity. Network is treated as an optional enhancement, not a dependency. Offline-first systems prioritize local consistency, durability, and resilience.

Non-Operational Content

Content that describes concepts, models, or structures without representing executable or deployable code. Non-operational content is used for documentation, reasoning, and architecture, not for runtime behavior.

Structural Dependency

A dependency that arises from the architecture or structure of the system, rather than runtime behavior. Structural dependencies define how components relate, organize, or rely on each other at the design level.

15. Index

A. Terms Index (A–Z)

Alphabetical list of all key technical terms used in the specification. Each term links to its definition or first appearance.

A

- **Atomic Write** — deterministic file replacement using temp file + rename
- **Append-Only** — write-only log structure without in-place mutation

C

- **Checksum** — integrity mechanism for detecting accidental corruption
- **Consistency** — predictable state across operations

D

- **Deterministic Execution** — same input → same output
- **Durability** — guarantees that written data persists

E

- **Explicit State** — no hidden or implicit state transitions

F

- **Framing** — structured message boundaries (length + payload + MAC)

H

- **HMAC** — authenticated integrity using a secret key

M

- **Minimal Surface** — smallest possible exposed interface
- **Message Envelope** — deterministic message container

N

- **Non-Operational Content** — conceptual, not executable

O

- **Offline-First** — system must operate without network
- **Oversize Protection** — rejecting payloads above limits

S

- **Schema Gate** — whitelist-based schema validation
- **Structural Dependency** — architecture-level dependency
- **Sync** — explicit persistence flush to storage

T

- **Tamper Detection** — detecting unauthorized modification
- **Typed Errors** — deterministic error categories

B. Definitions Index (Conceptual + Technical)

List of all formal definitions introduced in the document.

- **Deterministic Execution** — predictable behavior independent of environment
- **Explicit State** — fully visible and controlled state
- **Minimal Surface** — reduced attack and complexity surface
- **Offline-First** — network-independent operation
- **Non-Operational Content** — descriptive, not executable
- **Structural Dependency** — architecture-level linkage
- **Message Envelope** — deterministic container for structured data
- **Schema Gate** — schema whitelist validation
- **Atomic Write** — safe file replacement

- **HMAC Integrity** — authenticated message integrity
- **Oversize Constraint** — maximum allowed payload size
- **Typed Error** — deterministic error classification

16. Literature

16.1 Core Security & Cryptography References

- [1] B. Schneier, *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, Wiley, 1996. (Referenca za kriptografske principe, HMAC, integritet poruka.)
- [2] National Institute of Standards and Technology (NIST), *FIPS 198-1: The Keyed-Hash Message Authentication Code (HMAC)*, 2008. (Primarni standard za HMAC koji se koristi u VNAGY message envelope.)
- [3] National Institute of Standards and Technology (NIST), *SP 800-57: Recommendation for Key Management*, 2020. (Referenca za sigurno upravljanje ključevima, relevantno za HMAC ključ koji se ne pohranjuje u kodu.)
- [4] National Institute of Standards and Technology (NIST), *SP 800-63-3: Digital Identity Guidelines*, 2017.

16.2 Deterministic Systems & State Models

- [5] L. Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System*, Communications of the ACM, 1978. (Temelj za determinističko izvršavanje i ordering model.)
- [6] L. Lamport, *The Part-Time Parliament*, ACM Transactions on Computer Systems, 1998. (Relevantno za determinističke logove i state replication, iako VNAGY nije consensus engine.)
- [7] M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002. (Referenca za explicit state, minimal surface, i clean architecture.)

16.3 Offline-First & Resilient System Design

[8] J. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017. (Najbolja moderna referenca za offline-first, durability, atomic writes, log-structured storage.)

[9] A. Wiggins, *The Offline First Manifesto*, 2013.

16.4 File Systems, Atomicity, and Persistence

[10] D. Hitz, J. Lau, M. Malcolm, *File System Design for an NFS Server Appliance*, USENIX, 1994. (Klasik za atomic rename i durability — relevantno za VNAGY append log + snapshot model.)

[11] D. Gifford, *Weighted Voting for Replicated Data*, ACM SIGOPS, 1979. (Relevantno za determinističke persistence modele.)

16.5 Software Engineering Standards

[12] ISO/IEC 25010:2011, *Systems and Software Quality Requirements and Evaluation (SQuaRE)*. (Referenca za minimal surface, maintainability, and structural dependency.)

[13] ISO/IEC 27001:2022, *Information Security Management Systems*. (Široki sigurnosni okvir, relevantan za VNAGY security posture.)

16.6 Non-Operational Content & Conceptual Architecture

[14] IEEE Std 1471-2000, *Recommended Practice for Architectural Description of Software-Intensive Systems*. (Referenca za conceptual, non-operational content i structural architecture.)

[15] P. Kruchten, *The 4+1 View Model of Architecture*, IEEE Software, 1995. (Relevantno za VNAGY conceptual architecture specification.)

16.7 Additional Recommended References (Optional)

[16] R. Fielding, *Architectural Styles and the Design of Network-Based Software Architectures*, 2000. (Relevantno za minimal surface i clean interface design.)

[17] E. Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994. (Korisno za structural dependency i modularnost.)

17. Intellectual Property & Licensing

17.1. Ownership

The VNAGY Secure-First Architecture, including all symbolic constructs, interaction rules, structural definitions, and conceptual models, is the exclusive intellectual property of the author.

All rights are reserved.

17.2. Permitted Use

The following actions are permitted:

- reading and personal study
- academic reference
- citation with attribution
- sharing the document in its original, unmodified form

17.3. Prohibited Use

The following actions are strictly prohibited:

- implementation of any construct, rule, or model
- derivation of operational logic from symbolic definitions
- reconstruction of algorithms, thresholds, weights, or detection semantics
- commercial use in any form
- modification, removal, or alteration of licensing terms

- redistribution of modified versions

17.4. Non-Operational Declaration

This document contains no operational details, no thresholds, no weights, no heuristics, no correlation logic, and no algorithmic semantics. All constructs are symbolic and conceptual. The model is non-computational and non-reconstructable by design.

17.5. License

VNAGY Extended License (PSDL-1.3)

VNAGY CC BY-NC 4.0 Extended License

© 2026 Viktorija Nađ — All rights reserved.

License reference:

<https://github.com/VNAGY-sec-ViktorijaN/VNAGY-Framework/tree/main/VNAGY/LICENSE>